

P2 Documentation and Code Project

You are viewing draft content
created with the use of Claude.AI



P2 Assembly Programming

A Human-Centered Approach to Parallel Processing

January 2026

Version 2.0

Tutorial Philosophy

Learn by doing, celebrate progress, have fun!

Code Block Colors:

- **Green** – Spin2
- **Yellow** – PASM2
- **Purple** – CORDIC
- **Blue** – Multi-COG
- **Red** – Antipattern

Teaching Elements:

- **Sidetracks** – deeper dives
- **Medicine Cabinet** – simpler alternatives
- **Your Turn** – hands-on exercises
- **Interludes** – stories & context

Contents

Copyright and License	8
Dedication	9
Acknowledgments	10
Preface: Welcome to the Journey	12
Chapter 1: Your First Spin	15
Why P2?	15
The Hook: Making Light	15
What's Really Happening	16
Sidetrack: The Clock Preamble	17
Let's Make It Better	18
Understanding COGs	18
Your Turn: Experiments	19
Sidetrack: Why Start at Address 0?	21
Common Gotchas	22
What We've Learned	22
Coming Up Next	22
Chapter 2: Architecture Safari	24
The Propeller Philosophy	24
COG Anatomy 101	25
Meet the Hub: The Meeting Place	26
Let's See COGs in Action	27
COG Communication: How They Talk	27
The Timer: Everyone Gets One	29
Why No Interrupts? (Usually)	29
Real-World Example: Parallel Sensors	30
Sidetrack: The Philosophy of Parallel	31
Common Gotchas	32
What We've Learned	32
Your Turn: Experiments	32

Coming Up Next	34
Chapter 3: Speaking PASM2	35
The Hook: One Instruction, Many Powers	35
Instruction Anatomy 101	35
The Basic Vocabulary	36
Flow Control: Jump!	38
Labels: Naming Your Places	41
Data in DAT Blocks: Your Program's Pantry	43
The Flags: C and Z (and Q!)	49
Special Instructions That Will Blow Your Mind	49
Real-World Example: Fast Memory Copy	50
Common Gotchas	51
Your Turn: Experiments	52
Sidetrack: Why PASM2 Is Different	53
What We've Learned	54
Coming Up Next	54
Chapter 4: The Hub Connection	55
The Hook: Instant Communication	55
Reading from Hub	55
Writing to Hub	55
The FIFO Pipeline	56
Real-World Example: Video Buffer	57
Chapter 5: Mathematics Unleashed	59
The Hook: Hardware Multiply	59
The Multiplication Revolution	59
Division Without Tears	59
64-Bit Operations	60
Real-World Example: Fixed-Point Math	60
Chapter 6: Flags and Decisions	61
The Hook: Any Instruction Can Be Conditional	61
The C and Z Flags	61
Complex Conditions	61
Skip Patterns - Conditional Blocks	62
Chapter 7: CORDIC Magic	63
The Hook: Rotate a Point in 3 Lines	63
What Just Happened?	63
The CORDIC Pipeline - Your Mathematical Assembly Line	64

Core CORDIC Operations	64
Real-World Example: Spinning a Sprite	66
Your Turn: CORDIC Experiments	67
Advanced CORDIC: The Pipeline Dance	68
CORDIC for Graphics	69
CORDIC for Audio	70
Common CORDIC Gotchas	71
What About QLOG, QEXP?	71
What We've Learned	72
Coming Up Next	72
Chapter 8: Basic I/O	73
The Hook: One Pin, Three Instructions, Infinite Possibilities	73
Understanding P2 Pins	73
Digital Output: Making Things Happen	74
Digital Input: Reading the World	75
Pin Timing: When Things Happen	75
Real-World Example: Button Debouncing	76
Bit-Banged Serial (The Basics)	76
Your Turn: I/O Experiments	77
Advanced Pin Control	79
Common I/O Gotchas	79
Timing Is Everything	79
Real-World Example: Servo Control	80
The Power of Simple	80
What We've Learned	80
A Note About Smart Pins	81
Coming Up Next	81
Chapter 9: Streaming Data	82
The Hook: 4KB in 4 Instructions	82
Block Transfers: The Power Move	82
The FIFO: Your Streaming Pipeline	82
Writing Through the FIFO	83
Real-World Example: Screen Buffer Clear	83
Streaming with the Streamer	84
FIFO and COG Execution	84
Advanced Streaming Techniques	85
Your Turn: Streaming Experiments	86
Common Streaming Gotchas	87
Performance Numbers	87

Real-World Example: Audio Buffer	88
What We've Learned	88
Coming Up Next	89
Chapter 10: Hub Execution	90
The Hook: Unlimited Code Space	90
COG vs Hub Execution: The Trade-offs	90
How Hub Execution Works	91
Real-World Example: Menu System	91
The Hub Execution FIFO	92
Mixing COG and Hub Code	93
Your Turn: Hub Execution Experiments	93
Advanced Hub Execution	94
Common Hub Execution Gotchas	96
Real-World Example: Command Parser	96
When to Use Hub Execution	97
What We've Learned	97
Coming Up Next	97
Chapter 11: Why No Interrupts?	98
The Hook: Interrupts Without Interrupts	98
The Interrupt Problem	99
The Propeller Solution	99
Real-World Example: Perfect Servo Control	100
“But P2 HAS Interrupts!”	101
The Event System: Better Than Interrupts	102
Interrupt Horror Stories	103
Your Turn: COG vs Interrupt Challenge	104
The Philosophy Deep Dive	105
When Interrupts Actually Make Sense	105
Common “But What About...” Questions	106
What We've Learned	106
Coming Up Next	106
Chapter 12: Optimization Mastery	107
The Hook: Double Your Speed with One Change	107
Understanding the Pipeline	107
Instruction Timing Basics	108
REP: The Speed Loop	108
SKIP: Conditional Execution on Steroids	109
Hub Access Optimization	109

The FIFO Fast Path	109
Parallel Operations	110
Real-World Example: Fast Memory Copy	110
Your Turn: Optimization Challenges	112
Advanced Techniques	113
Common Optimization Gotchas	114
Profiling and Measurement	114
What We've Learned	114
Coming Up Next	115
Chapter 13: LUT Memory - Your Private Lookup Table	116
The Hook: A Lookup Table in 3 Cycles	116
Why Another Memory?	116
Reading and Writing the LUT	117
LUT Sharing Between COGs	118
Practical Examples	119
LUT with the Streamer	120
Common Gotchas	121
Medicine Cabinet	121
Your Turn	122
Chapter 14: Smart Pins Orientation	124
The Hook: A UART in 4 Lines	124
What Are Smart Pins, Really?	124
The Universal Smart Pin Pattern	124
The Core Instructions	125
Understanding the IN Flag	126
Common Smart Pin Modes	127
Configuration Values Demystified	128
Common Gotchas	129
Medicine Cabinet	130
Your Turn	130
Chapter 15: Event-Driven Programming	132
The Hook: No More Polling Loops	132
Why Events Matter	132
The Four Selectable Events	132
Configuring an Event	133
Timer Events	136
Waiting vs Polling	136
Multiple Events	137

Practical Examples	137
ATN - Inter-COG Events	138
Common Gotchas	139
Medicine Cabinet	140
Your Turn	141
Chapter 16: Multi-COG Orchestration	142
The Hook: A Complete System in 8 COGs	142
Communication Patterns	143
Synchronization Techniques	145
Real-World Example: Robot Controller	145
Your Turn: Multi-COG Project	148
Design Principles for Multi-COG Systems	149
Common Multi-COG Gotchas	150
What We've Learned	150
Your Journey Continues	150
Chapter Summary	150
Epilogue: The Journey Forward	151
Further Reading	152
Appendix A: Platform Comparison	153
The Landscape	153
What Makes P2 Different	153
Coming From ARM/STM32	154
Coming From ESP32	155
Coming From Arduino/AVR	155
When P2 Is the Right Choice	156
Platform Trade-offs	156
Community Resources	157
The P2 Hardware Ecosystem	157
Summary	158
Index	159

Copyright and License

Copyright © 2025-2026 Iron Sheep Productions, LLC and Parallax Inc.

This work is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License (CC BY-SA 4.0).

You are free to:

- **Share** — copy and redistribute the material in any medium or format
- **Adapt** — remix, transform, and build upon the material for any purpose, even commercially

Under the following terms:

- **Attribution** — You must give appropriate credit, provide a link to the license, and indicate if changes were made
- **ShareAlike** — If you remix, transform, or build upon the material, you must distribute your contributions under the same license

To view the full license, visit: <https://creativecommons.org/licenses/by-sa/4.0/>

Trademarks

Propeller, Propeller 2, P2, Spin, and the Parallax logo are trademarks of Parallax Inc.

Disclaimer

The information in this manual is subject to change without notice. While every effort has been made to ensure accuracy, the authors and publishers assume no responsibility for errors or omissions, or for damages resulting from the use of the information contained herein.

Dedication

To deSilva — *Whose legendary P1 assembly tutorial taught a generation of programmers that assembly language could be approachable, enjoyable, and even fun. Your unique voice—combining technical precision with human warmth—showed us that great documentation teaches not just the mind, but speaks to the spirit of discovery.*

To the Propeller Community — *Who have spent countless hours exploring, documenting, and sharing their knowledge. From the early P1 pioneers to today’s P2 innovators, your collective wisdom makes this manual possible.*

To Future Makers — *May you find in these pages the same joy of discovery that we experienced. The Propeller 2 is more than a microcontroller—it’s an invitation to think differently about computing. Welcome to the journey.*

“The best way to predict the future is to invent it.” — Alan Kay

Acknowledgments

This manual stands on the shoulders of giants. We gratefully acknowledge:

Primary Contributors

deSilva - For creating the gold standard of microcontroller documentation with the P1 Assembly Tutorial. Your pedagogical approach, combining technical depth with human empathy, remains unmatched. This manual attempts to honor your legacy while adapting to the P2's capabilities.

Iron Sheep Productions LLC (Stephen M Moraco) - For extensive P2 documentation efforts, community tools, and the vision of creating an AI-optimized knowledge base. Your systematic approach to extracting and organizing P2 knowledge made this comprehensive manual possible.

Chip Gracey - Creator of the Propeller architecture. Thank you for giving us a microcontroller that thinks differently and challenges us to do the same.

Community Contributors

The Parallax Forums Community - Your questions, answers, code examples, and endless experimentation have created a living knowledge base that no single author could match.

Early P2 Adopters - Who dealt with evolving documentation, changing specifications, and still produced amazing projects that showed us what was possible.

Technical Reviewers

Special thanks to those who reviewed drafts, tested code examples, and provided invaluable feedback:

- The P2 Documentation Team at Parallax
- Community members who beta-tested examples
- Everyone who reported errors and suggested improvements

Inspiration

The MIT AI Lab - For showing us that technical documentation can have personality

Donald Knuth - For proving that programming texts can be literature

The Demoscene Community - For pushing hardware beyond its limits and inspiring us to do the same

Production Notes

This manual was created using:

- Knowledge extracted from official Parallax technical documentation and OBEX (Object Exchange) community contributions
- AI-assisted content generation trained on deSilva’s writing style
- Community validation and real-world testing
- A commitment to making parallel processing accessible to everyone

“If I have seen further, it is by standing on the shoulders of giants.” — Isaac Newton

Any errors, omissions, or dad jokes that fell flat are entirely the responsibility of the authors, not our distinguished contributors.

Preface: Welcome to the Journey

Well, here we are! You're about to embark on a journey into the heart of the Propeller 2, and I promise you, it's going to be quite different from what you might expect.

A Different Kind of Processor

The Propeller 2 isn't just another microcontroller. Oh no, it's something far more interesting. Imagine, if you will, eight independent processors (we call them COGs) all working together in perfect harmony, sharing a common memory space, yet each running their own programs at full speed. No interrupts fighting for attention, no complex priority schemes, just eight brains working in parallel.

And if you think this sounds terribly complicated, you're probably right... but here's the secret: it's actually simpler than traditional architectures once you understand the philosophy.

About This Manual

This manual follows in the footsteps of deSilva's legendary P1 tutorial. What does that mean? It means we're going to:

1. **Start with working code** - You'll be blinking LEDs before you know what hit you
2. **Learn by doing** - Theory is important, but nothing beats hands-on experience
3. **Have some fun** - Yes, assembly language can actually be enjoyable!
4. **Be honest about complexity** - When something is hard, we'll admit it and then show you how to handle it

Who Is This For?

Are you a complete beginner to assembly language? Welcome! We'll take good care of you.

Are you a grizzled veteran who's been writing assembly since the 6502? Welcome! The P2 will still surprise you.

Are you somewhere in between? Perfect! This is exactly where you want to be.

The only requirement is curiosity and a willingness to think a bit differently about how computers can work.

How to Use This Manual

The Fast Track — If you're impatient (and who isn't?), jump straight to Chapter 1. Get that LED blinking. Feel the satisfaction. Then come back here when you're ready for more.

The Scenic Route — Read the chapters in order. Each builds on the previous one, and I've hidden little gems of knowledge throughout that will make later chapters easier.

The Reference Approach — Already know what you're looking for? The table of contents and index are your friends. The appendices contain every instruction, every Smart Pin mode, every CORDIC operation.

What Makes the P2 Special?

Let me count the ways:

- **8 symmetric COGs** - No master/slave relationships, all COGs are equal
- **64 Smart Pins** - Each pin has its own processor for I/O operations
- **CORDIC engine** - Hardware trigonometry and coordinate transformations
- **Hardware multiply/divide** - Finally! Real math at hardware speed
- **512KB of RAM** - Shared by all COGs with deterministic access timing
- **No interrupts** - Well, actually there are interrupts, but we'll talk about why you probably don't want them

A Note on Our Approach

The best technical documentation remembers you're human. You'll get frustrated. You'll make mistakes. Your code won't work the first time (or the second, or sometimes even the third).

That's normal. That's learning. And that's why this manual provides plenty of "medicine" along the way - simpler alternatives, working examples, and the occasional moment of levity to keep your spirits up.

The deSilva Spirit

Throughout this manual, you'll encounter the teaching spirit of deSilva. When you see phrases like:

- "Well, ..." - We're about to correct a common assumption
- "Uff!" - We just got through something complex
- "Have Fun!" - We mean it, this stuff is actually enjoyable

These aren't just quirks; they're signals that we remember you're human and we're on this journey together.

Ready?

Take a deep breath. Pour yourself your favorite beverage. Open your development environment.

Let's make some magic happen with the Propeller 2!

“The Propeller architecture is based on the simple idea that the best way to avoid the complexity of interrupts is to have enough processors that you don't need them.”

— Chip Gracey, creator of the Propeller

Turn the page, and let's blink that LED! →

Chapter 1: Your First Spin

Let's blink an LED and change your life

Why P2?

Before we dive into code, let me tell you why you're in for something different.

If you've fought with interrupt priority conflicts on an ARM, watched your timing jitter because of cache misses, or discovered that the UART you need is only available on pins you're already using... well, the P2 was designed by someone who got tired of those problems too.

Here's the P2 philosophy in a nutshell:

Instead of one processor fighting with interrupts, you get eight complete, identical processors (COGs) that run truly in parallel. Your serial handler never delays your motor control. Your sensor sampling never misses a deadline. Each task owns its own processor.

Instead of fixed peripherals, every one of the 64 pins contains its own programmable state machine. Any pin can become a UART, PWM output, quadrature encoder, ADC - whatever you need, wherever you need it.

Instead of timing that depends on cache luck, the hub memory has deterministic access. Your timing loops work the same way every time.

Instead of calling math libraries, there's a hardware CORDIC that computes sine, cosine, and arctangent in exactly 55 clocks. Every time.

Does this mean P2 is perfect for everything? Of course not. But if your projects involve multiple real-time tasks, precise timing, video or audio generation, or just running out of peripheral pins - you're in the right place.

For a full comparison to ARM, ESP32, Arduino, and PIC platforms, see Appendix A. But you probably want to blink that LED first, don't you?

The Hook: Making Light

I know you're absolutely crazy to have your first instruction executed, so let's not waste any time. Here's a complete PASM2 program that blinks an LED on pin 56 (that's the built-in LED on the P2 Eval board):

```
1  CON
2    _clkfreq = 200_000_000      ' 200 MHz system clock
3  DAT
4  ' LED Blinker - Your first PASM2 program!
5      ORG      0                ' Start at COG address 0
6      DRVH     #56              ' Drive pin 56 high (LED on)
7      WAITX    ##50_000_000    ' Wait 0.25 seconds at 200MHz
8      DRVL     #56              ' Drive pin 56 low (LED off)
9      WAITX    ##50_000_000    ' Wait 0.25 seconds
10     JMP      $$-4             ' Jump back 4 longs (addresses)
```

That's it! Five instructions and you have a blinking LED. Load this into any COG and watch the magic happen.

What's Really Happening

Well, now that you've seen it work (you did try it, right?), let's talk about what's actually going on here.

The Instructions Decoded

ORG 0 - This tells the assembler to start placing code at COG address 0. Every COG has its own private 512 longs (2KB) of memory, and execution always starts at address 0 when a COG is loaded.

DRVH #56 - This drives pin 56 high (3.3V). The 'h' means high. The '#' means we're using an immediate value (the actual number 56) rather than the contents of register 56. One instruction, and your LED is on!

WAITX ##50_000_000 - This waits for 50 million clock cycles. At 200 MHz, that's 0.25 seconds. Notice the double '##'? That means this is a 32-bit immediate value. Single '#' only gives us 9 bits.

DRVL #56 - Drive low. LED off. You get the pattern.

JMP \$\$-4 - Jump back 4 longs. The '\$' means "current address", so '\$-4' means "4 addresses back from here" (each instruction is one long). Infinite loop achieved!

But Wait, There's More!

"Hold on," you might say, "how does this even get into the COG?"

Ah, excellent question! In the real world, you'd typically launch this from Spin2 (the high-level language) like this:


```
1 CON
2   _clkfreq = 200_000_000   ' 200 MHz system clock
3 PUB main()
4   COGINIT(COGEXEC_NEW, @blink_code, 0)   ' Start PASM2 in new COG
5   repeat   ' Keep the main COG alive
6 DAT
7   ORG      0
8 blink_code
9   DRVH     #56
10  WAITX    ##50_000_000
11  DRVL     #56
12  WAITX    ##50_000_000
13  JMP      #-$-4
```

The `COGINIT` instruction loads your PASM2 code from hub memory into a fresh COG and starts it running. Meanwhile, your Spin2 code keeps running in its own COG. You now have parallel processing!

Sidetrack

The Clock Preamble

Notice the `CON` section at the top of that example? Every P2 program needs to configure its system clock:

```
1 CON
2   _clkfreq = 200_000_000   ' 200 MHz system clock
```

This tells the P2 to run at 200 MHz using your board's crystal oscillator. Without it, the chip runs at a sluggish ~20 MHz on its internal RC oscillator—and timing-dependent code (including `DEBUG` output) won't behave as expected.

At 200 MHz with most instructions taking 2 clocks, each COG executes approximately 100 million instructions per second (100 MIPS). With 8 COGs running in parallel, that's 800 MIPS of total processing power—and that's before Smart Pins start handling I/O autonomously.

From here on, we'll omit this preamble from examples to keep them focused on the concept being taught. When you create your own files, always include it at the top before your `PUB` or `DAT` sections.

Let's Make It Better

The blinker works, but it's a bit rigid, isn't it? What if we want to change the blink rate? Let's use a register:

```

1          ORG      0
2
3          MOV      delay, ##50_000_000    ' Set delay to 0.25 sec
4                                           ' (at 200 MHz)
5
6 .blink   DRVH      #56                    ' LED on
7          WAITX    delay                  ' Wait
8          DRVL      #56                    ' LED off
9          WAITX    delay                  ' Wait
10         JMP      #.blink                 ' Repeat forever
11
12 delay   LONG      0                      ' Storage for delay value

```

Uff! Look at that - we're using a register now! The `MOV` instruction copies our delay value into a register (which we cleverly named 'delay'). Now we can change the blink rate by modifying just one value.

A note on terminology: P2 documentation often uses “register” to refer to any long in COG RAM. Unlike ARM or x86 where registers are a small, special set (R0-R15, EAX, etc.), every COG RAM location can be used as a general-purpose register. However, the last 16 locations (496-511) have special functions: addresses 496-503 are dual-purpose (usable as RAM if interrupts aren't used), and 504-511 are special-purpose registers (PTRA, PTRB, DIRA, DIRB, OUTA, OUTB, INA, INB). When you see “register” in P2 context, think “COG RAM location.”

Understanding COGs

Here's something important: each COG is a complete processor with its own memory. When we loaded our blink program, it was copied from hub memory into COG memory. The COG then executes it independently, without any further connection to hub memory (unless we explicitly read or write to it).

Think of it like this:

- **Hub memory** is the meeting place (512KB shared by all)
- **COG memory** is private workspace (2KB per COG)
- Loading a COG is like making a photocopy - the COG gets its own copy to run

This is why our blinker keeps running even after the Spin2 code that launched it goes into an infinite repeat loop. The COG is independent!

Your Turn: Experiments

Now for the fun part. Try these modifications:

Experiment 1: Different Patterns

Make the LED blink in a pattern: short-short-long (like SOS):

```

1      ORG      0
2
3      MOV      short, ##20_000_000    ' 0.1 second (at 200 MHz)
4      MOV      long_d, ##60_000_000   ' 0.3 seconds (at 200 MHz)
5
6 .pattern DRVH    #56                  ' Short pulse 1
7      WAITX    short
8      DRVL     #56
9      WAITX    short
10     DRVH     #56                    ' Short pulse 2
11     WAITX    short
12     DRVL     #56
13     WAITX    short
14     DRVH     #56                    ' Long pulse
15     WAITX    long_d
16     DRVL     #56
17     WAITX    long_d
18     JMP      #.pattern
19
20 short  LONG   0
21 long_d LONG   0

```

Experiment 2: Multiple LEDs

Blink LEDs on pins 56 and 57 alternately:

```

1      ORG      0
2
3  .loop  DRVH    #56          ' LED 56 on
4         DRVL    #57          ' LED 57 off
5         WAITX   ##50_000_000 ' 0.25 sec at 200 MHz
6         DRVL    #56          ' LED 56 off
7         DRVH    #57          ' LED 57 on
8         WAITX   ##50_000_000 ' 0.25 sec at 200 MHz
9         JMP     #.loop

```

Experiment 3: Fading (Advanced)

This one's a bit tricky - we'll use PWM to fade the LED:

```

1      ORG      0
2
3      WRPIN    ##P_PWM_TRIANGLE, #56 ' Configure pin 56 for PWM
4      WXPIN    ##$100, #56          ' Set period to 256
5      DIRH     #56                  ' Enable the pin
6
7  .fade  WYPIN   level, #56          ' Set duty cycle
8         WAITX   ##100_000          ' Small delay
9         ADD     level, #1           ' Increment brightness
10        AND     level, #$FF         ' Wrap at 256
11        JMP     #.fade
12
13 level  LONG    0

```

Don't worry if the PWM example seems complex - we'll cover Smart Pins in detail in Chapter 8!

The Medicine Cabinet

Feeling overwhelmed? Here's the simplified prescription:

Minimum viable blinker - Just 3 instructions:

```
1 .loop    DRVNOT    #56          ' Toggle pin 56
2          WAITX     ##50_000_000 ' 0.25s at 200MHz
3          JMP       #.loop       ' Repeat
```

The `DRVNOT` instruction toggles a pin - if it's high, make it low; if it's low, make it high. Sometimes simpler is better!

Sidetrack

Why Start at Address 0?

You might wonder why COG code always starts at address 0. It's actually quite elegant:

When a COG is started with `COGINIT`, the hardware:

1. Stops the COG (if it was running)
2. Copies 512 longs from hub to COG memory (addresses 0-511)
3. Starts execution at COG address 0

This means every COG program starts fresh, with a clean slate. No residual state, no confusion. It's like each COG gets a fresh brain transplant every time it starts!

The last 16 longs (addresses 496-511) have special functions: 496-503 are dual-purpose (usable as RAM if interrupts not used), and 504-511 are special-purpose registers (`PTRA`, `PTRB`, `DIRA`, `DIRB`, `OUTA`, `OUTB`, `INA`, `INB`). We'll explore these later.

Common Gotchas

Before we move on, let me save you some debugging time:

1. **Forgetting the ##** - Using `WAITX #25_000_000` will NOT wait for 0.25 seconds! Single `#` only allows values up to 511.
2. **Wrong pin number** - The P2 Eval board's LEDs are on pins 56-63. The P2 Edge module might have different assignments.
3. **Clock setup required** - P2 boots at ~20MHz (internal RC oscillator). Most programs configure 200MHz with a crystal. Our examples assume 200MHz - adjust `WAITX` values if your clock differs.
4. **COG already running** - If you `COGINIT` to a specific COG that's already running something else, it will be stopped and replaced. Use `COGEXEC_NEW` to automatically find a free COG.

What We've Learned

Let's celebrate what you've accomplished:

- Written your first PASM2 program
- Controlled hardware (LED) directly
- Used immediate values (`#` and `##`)
- Created loops with `JMP`
- Understood COG independence
- Modified code for different patterns

That's quite a lot for Chapter 1!

Coming Up Next

In Chapter 2, we'll take our "Architecture Safari" and explore:

- How 8 COGs really work together
- The hub memory system and the "egg beater"
- Why the P2 doesn't need interrupts
- How to make COGs talk to each other

But for now, enjoy your blinking LED. You've just taken your first step into parallel processing!

Have Fun! And remember, every expert was once a beginner who kept their LED blinking when everyone else gave up.

Continue to Chapter 2: Architecture Safari →

Chapter 2: Architecture Safari

Eight brains are better than one

The Propeller Philosophy

Before we dive into the technical details, let's talk philosophy. Why would anyone design a micro-controller with eight processors?

The answer is beautifully simple: to avoid complexity.

"Wait," you might say, "eight processors sounds MORE complex, not less!"

Well, consider the traditional approach:

- One processor trying to do everything
- Interrupts constantly breaking your flow
- Priority levels to juggle
- Context switching overhead
- Race conditions and timing nightmares

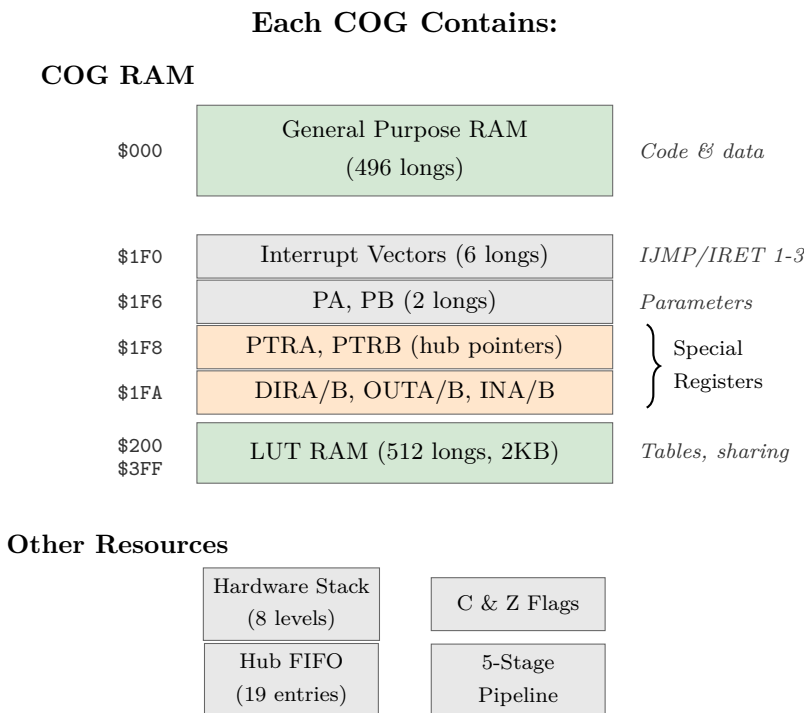
Now consider the Propeller way:

- Eight processors, each doing one thing well
- No interrupts needed (why interrupt when you have a dedicated processor?)
- No priorities (all COGs are equal)
- Deterministic timing (you know EXACTLY when things happen)
- True parallel processing (not time-slicing)

It's like the difference between one stressed-out juggler trying to keep eight balls in the air versus eight relaxed people each tossing one ball. Which seems simpler?

COG Anatomy 101

Let's dissect a COG and see what makes it tick:



But here's the beautiful part: COGs are identical. There's no "master" COG or "special" COG. Any COG can do anything any other COG can do. Democracy in silicon!

The 512-Long Limit

Each COG has exactly 512 longs (2048 bytes) of memory. The first 496 longs are yours to use for code and data. The last 16 are special registers (but not like P1 - we'll get to that).

"Only 496 instructions?" you might cry. "That's tiny!"

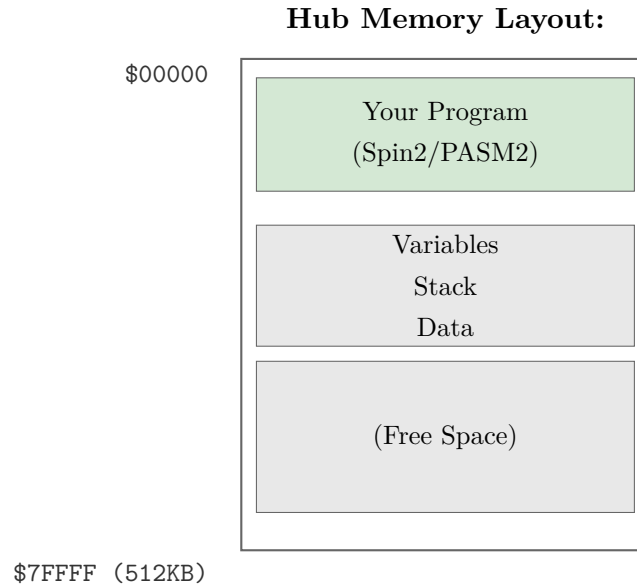
Well, yes and no. Remember:

1. PASM2 instructions are powerful - one instruction often does what takes several in other processors
2. You have EIGHT of these COGs
3. There's hub execution mode for larger programs (Chapter 10)
4. Most real-time tasks fit easily in 496 instructions

Think of it like haiku - the constraint forces elegance.

Meet the Hub: The Meeting Place

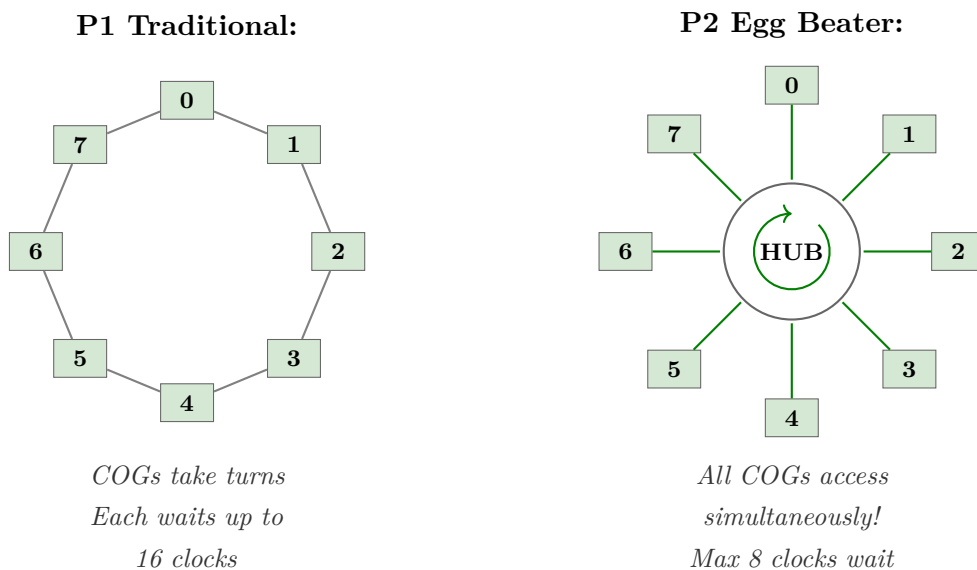
The hub is where COGs come together. It's 512KB of RAM shared by all COGs, and it's where the magic of cooperation happens.



The Egg Beater Revolution

Now here's where P2 gets clever. In P1, COGs took turns accessing the hub in a round-robin fashion. If you missed your slot, you waited for the wheel to come around again.

P2 uses what we call the “egg beater” model. Imagine eight beaters (COGs) all whipping through the same bowl (hub) simultaneously, but their paths are cleverly arranged so they never collide:



The practical result? Hub access is MUCH faster and more predictable. Instead of waiting up to 16 clocks (P1), you wait at most 8 clocks (P2), and often less if you align your accesses properly.

Let's See COGs in Action

Here's a simple demonstration of multiple COGs working together:

```

1  ' Multi-COG LED Pattern Demo
2  PUB main() | i
3      repeat i from 0 to 3
4          COGINIT(COGEXEC_NEW, @cog_code, 56 + i) ' Start 4 COGs
5      repeat ' Main COG just watches
6  DAT
7      ORG      0
8  cog_code
9      RDLONG   pin_num, ptra          ' Get pin number from hub
10     .loop    DRVNOT pin_num          ' Toggle our LED
11             SHL      pin_num, #24    ' Pin number to bits 24-31
12             OR       pin_num, ##10_000_000 ' Combine with delay
13             WAITX    pin_num          ' Wait (varies per COG!)
14             SHR      pin_num, #24    ' Restore pin number
15             JMP      #.loop
16 pin_num LONG 0

```

What's happening here:

1. The main Spin2 code starts 4 COGs
2. Each COG gets a different pin number (56, 57, 58, 59)
3. Each COG blinks its LED at a slightly different rate
4. All four LEDs blink independently and simultaneously!

COG Communication: How They Talk

COGs are independent, but they're not isolated. They can communicate through hub memory:

Method 1: Simple Variables

```

1 ' COG 1: Writer
2     MOV     value, #42
3     WRLONG  value, ##$1000 ' Write to hub address $1000
4 ' COG 2: Reader
5     RDLONG  result, ##$1000 ' Read from hub address $1000

```

Method 2: Locks (When It Matters)

When multiple COGs might write to the same location, we need locks:

```

1 ' Get a lock
2 .try_lock
3     LOCKTRY lock_id wc      ' Try to get lock
4     if_c JMP     #.try_lock ' Keep trying if failed
5     ' Critical section - we have the lock!
6     RDLONG  value, ##shared_addr
7     ADD     value, #1
8     WRLONG  value, ##shared_addr
9     LOCKREL lock_id        ' Release the lock
10 lock_id LONG    0          ' Lock 0-15

```

Method 3: Mailboxes (Elegant)

A mailbox is just a hub location where COGs leave messages:

```

1 ' COG A: Leave a message
2     WRLONG  message, ##mailbox
3
4 ' COG B: Check for messages
5 .check RDLONG  data, ##mailbox wz
6     if_z JMP     #.check      ' Keep checking if empty
7     WRLONG  #0, ##mailbox    ' Clear mailbox
8     ' Process the message in 'data'

```

The Timer: Everyone Gets One

Each COG has its own 64-bit timer, always counting system clocks. This is incredibly useful:

```

1  ' Method 1: Simple delay
2      GETCT    start_time
3      ADDCT1   start_time, ##1_000_000
4      WAITCT1                      ' Wait exactly 1,000,000 clocks
5  ' Method 2: Periodic events
6      GETCT    time
7  .loop  ADDCT1  time, ##10_000_000
8      WAITCT1                      ' Wait for next 10M clock interval
9      DRVNOT   #56                  ' Toggle LED
10     JMP      #.loop               ' Perfectly periodic!

```

The beauty? Each COG's timer is independent. No shared resource conflicts!

Why No Interrupts? (Usually)

Here's a controversial P2 feature: it HAS interrupts, but you probably shouldn't use them. Why?

Because with 8 COGs, you don't need interrupts! Instead of interrupting important work, just dedicate a COG to monitoring whatever would have triggered the interrupt:

```

1  ' Traditional (with interrupts):
2  ' Main code runs, gets interrupted, handles event, returns
3  ' Propeller way:
4  ' COG 1: Main code runs uninterrupted
5  ' COG 2: Watches for event continuously
6  pin_watcher
7      TESTP    #BUTTON_PIN wc
8      if_c JMP    #button_pressed
9      JMP      #pin_watcher
10
11 button_pressed
12      WRLONG   ##1, ##button_flag ' Signal other COGs
13      JMP      #pin_watcher

```

No interrupt latency, no context switching, no priority inversion. Just dedicated, deterministic monitoring.

Real-World Example: Parallel Sensors

Let's read four different sensors simultaneously:

```
1  ' Each COG runs this with different parameters
2  sensor_reader
3      RDLONG  sensor_pin, ptra[0]      ' Get pin assignment
4      RDLONG  hub_addr, ptra[1]        ' Where to store results
5
6  read_loop
7      ' Read sensor (simplified - real sensors need protocols)
8      TESTP   sensor_pin wc
9      RCL     value, #1                ' Accumulate bits
10
11     ' Every 32 reads, store to hub
12     INCMOD  counter, #31 wc
13     if_c WRLONG value, hub_addr
14     if_c ADD  hub_addr, #4
15
16     JMP     #read_loop
17 sensor_pin LONG 0
18 hub_addr   LONG 0
19 value      LONG 0
20 counter    LONG 0
```

Four COGs running this code = four sensors being read truly simultaneously. Try doing that with a single processor and interrupts!

The Medicine Cabinet

Feeling overwhelmed by all this parallel processing? Here's your prescription:

Start simple: Use just one or two COGs at first

```
1 ' Just two COGs - main program and one helper
2 PUB main()
3     COGINIT(COGEXEC_NEW, @helper, 0)
4     ' Your main code here
```

Debug one COG at a time: Get each COG working alone before combining

```
1 ' Test COG in isolation first
2 debug_cog
3     DRVH     #MY_DEBUG_LED ' Visual confirmation it's running
4     ' Your actual code here
```

Use Spin2 for coordination: Let the high-level language handle the complex stuff

```
1 ' Spin2 manages COGs, PASM2 does the real-time work
2 PUB orchestrator()
3     startSensorCog(0)
4     startMotorCog(1)
5     startCommsCog(2)
6     ' Spin2 coordinates, PASM2 executes
```

Sidetrack

The Philosophy of Parallel

The Propeller's design philosophy comes from a simple observation: in the real world, things happen in parallel, not in sequence.

Consider your car:

- The engine runs continuously
- The radio plays independently
- The climate control maintains temperature
- The dashboard updates displays
- The ABS monitors wheel speed

These aren't taking turns - they're all happening simultaneously. The Propeller models

this reality directly. Instead of one processor frantically time-slicing between tasks, you have eight processors each focused on their job.

It's not just different - it's more natural.

Common Gotchas

Save yourself some debugging time:

1. **COG RAM is copied, not shared** - Changes in COG RAM don't affect hub RAM unless you explicitly write them back
2. **COGs start at 0** - Always! Your code better be there.
3. **Hub addresses are byte addresses** - COG addresses are long addresses. Don't mix them up!

```
1    RDLONG  value, ##$1000  ' Reads from hub byte address $1000
2    MOV     value, $100      ' Moves from COG long address $100 (256)
3    ' Note: COG RAM is only 512 longs ($000-$1FF)!
```

4. **PTRA/PTRB are your friends** - These special registers make hub access much easier
5. **COGs are truly independent** - Stopping one COG doesn't affect others (unless they're waiting for it)

What We've Learned

Look at what you now understand:

- Why eight processors is simpler than one with interrupts
- How COGs are structured and limited
- The hub memory system and egg beater access
- Multiple ways for COGs to communicate
- Why interrupts are usually unnecessary
- How to think in parallel

Your Turn: Experiments

Experiment 1: COG Counter

Start COGs to increment different hub locations. With COGEXEC_NEW, the loop will start up to 7 new COGs (since COG 0 runs Spin2):

```

1  PUB main() | i
2      repeat i from 0 to 7
3          COGINIT(COGEXEC_NEW, @counter, $1000 + (i * 4))
4      repeat
5          ' Monitor the counters in hub RAM
6
7  DAT
8      ORG      0
9  counter RDLONG  hub_ptr, ptra
10 .loop  RDLONG  value, hub_ptr
11      ADD      value, #1
12      WRLONG   value, hub_ptr
13      WAITX    ##1_000_000
14      JMP      #.loop
15
16 hub_ptr LONG    0
17 value  LONG    0

```

Experiment 2: Parallel Pattern

Make 8 LEDs display a moving pattern, with each COG controlling one LED:

```

1  ' Each COG gets LED pin in ptra
2      ORG      0
3      RDLONG   pin, ptra
4      RDLONG   delay, ptrb      ' Different delay per COG
5
6  .flash  DRVH   pin
7      WAITX    delay
8      DRVL     pin
9      WAITX    delay
10     SHL      delay, #1          ' Double the delay
11     CMP      delay, ##100_000_000 wcz
12     if_a MOV  delay, ##1_000_000 ' Reset if too long

```

13

JMP

#.flash

Coming Up Next

In Chapter 3, “Speaking PASM2”, we’ll dive deep into the instruction set:

- The anatomy of an instruction
- Conditional execution that will blow your mind
- Math operations that actually make sense
- Why PASM2 is unlike any assembly you’ve used

But for now, appreciate what you’ve learned: you understand the Propeller’s parallel philosophy. That’s not just technical knowledge - it’s a new way of thinking about computing.

Have Fun! Remember, parallel processing isn’t harder - it’s different. And different can be wonderful.

Continue to Chapter 3: Speaking PASM2 →

Chapter 3: Speaking PASM2

Learning the native tongue

The Hook: One Instruction, Many Powers

Look at this single PASM2 instruction:

```
1      ADD      value, #1 wc
```

This one line:

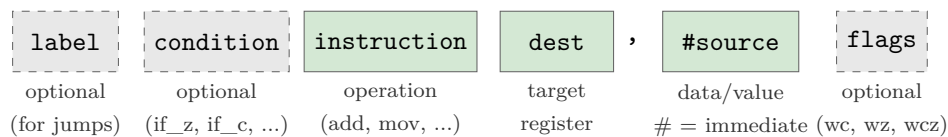
- Adds 1 to ‘value’
- Optionally sets the carry flag
- Executes in exactly 2 clock cycles
- Can be conditional
- Can even modify itself!

In most processors, that would take multiple instructions. In PASM2, it’s just one. Let’s learn to speak this powerful language.

Instruction Anatomy 101

Every PASM2 instruction follows the same basic pattern:

PASM2 Instruction Format:



Let’s dissect a real instruction:

Example: `if_z add total, #10 wc`



The Basic Vocabulary

Moving Data Around

The MOV family - your bread and butter:

```

1  ' Basic move
2      MOV      dest, source      ' dest = source
3      MOV      x, #42           ' x = 42 (immediate)
4      MOV      x, ##70000       ' x = 70000 (32-bit immediate)
5  ' But wait, there's more!
6      mvn      dest, source      ' dest = NOT source (inverted)
7      ABS      dest, source      ' dest = |source|
8      NEG      dest, source      ' dest = -source
9  ' And the mind-blowing ones
10     ALTD     dest, source      ' Modify NEXT inst's dest field!
11     ALTS     dest, source      ' Modify NEXT inst's source field!

```

Well, that escalated quickly! Don't worry about ALTD/ALTS yet - just know they exist and they're amazing.

Math Without Tears

P2 has hardware multiply and divide. Let that sink in. Hardware. Multiply. And. Divide.

```

1  ' Addition and subtraction
2      ADD      x, y              ' x = x + y
3      SUB      x, y              ' x = x - y
4      ADDS     x, y              ' Signed add
5      SUBS     x, y              ' Signed subtract
6  ' The revolution: hardware multiply!
7      MUL      x, y              ' x = x * y (low 32 bits)
8      MULS     x, y              ' Signed multiply
9
10 ' And hardware divide!
11     QDIV     x, y              ' Start division x/y
12     GETQX    result            ' Get quotient
13     GETQY    remainder         ' Get remainder

```

Here's a complete multiply example:

```

1  ' Simple 16x16->32 multiply (2 clocks)
2      MOV    x, #123
3      MOV    y, #456
4      MUL    x, y          ' Result: 123 * 456 = 56088 in x
5      ' Uses lower 16 bits of each operand!
6  ' For full 32x32->64 multiply, use CORDIC:
7      QMUL    x, y          ' Start multiply (uses full 32 bits)
8      ' ... 55 clocks of other work ...
9      GETQX   low           ' Lower 32 bits of result
10     GETQY   high          ' Upper 32 bits of result

```

Uff! In the old days, we'd write loops for this. Now hardware does it!

Logic Operations

Your Boolean friends:

```

1      AND    x, mask        ' x = x AND mask
2      OR     x, bits         ' x = x OR bits
3      XOR    x, toggle      ' x = x XOR toggle
4      NOT    x              ' x = NOT x (XOR with $FFFFFFFF)
5
6  ' Bit manipulation
7      BITL    x, #5          ' Clear bit 5 of x
8      BITH    x, #5          ' Set bit 5 of x
9      BITNOT  x, #5          ' Toggle bit 5 of x
10     TESTB   x, #5 wc        ' Test bit 5, result in C flag

```

Shifting and Rotating

Moving bits around:

```

1      SHL    x, #3           ' Shift left 3 bits
2      SHR    x, #3           ' Shift right 3 bits
3      SAR    x, #3           ' Arithmetic shift right (signed)
4      ROL    x, #3           ' Rotate left 3 bits
5      ROR    x, #3           ' Rotate right 3 bits
6
7  ' Variable shifts (amount in register)

```

```

8          SHL      x, y          ' Shift x left by y bits
9
10 ' Fancy ones
11          REV      x            ' Reverse bit order (!! )
12          MERGEB   x            ' Merge bytes (AABBCCDD -> ABCDABCD)

```

Flow Control: Jump!

Unconditional Jumps

```

1          JMP      #target      ' Jump to target
2          JMP      target      ' Jump to address in target register
3
4 ' Relative jumps
5          JMP      $$-4         ' Jump back 4 longs (addresses)
6          JMP      $$+8         ' Jump forward 8 instructions

```

Conditional Execution (The Magic)

Here's where PASM2 gets beautiful. ANY instruction can be conditional:

```

1 if_z     ADD      x, #1        ' Only add if Z flag set
2 if_nz    ADD      x, #1        ' Only add if Z flag clear
3 if_c     ADD      x, #1        ' Only add if C flag set
4 if_nc    ADD      x, #1        ' Only add if C flag clear

```

The basic conditions:

Condition	Meaning
if_z	If Z flag set (result was zero)
if_nz	If Z flag clear (result not zero)
if_c	If C flag set (carry/borrow occurred)
if_nc	If C flag clear
if_c_and_z	If both C and Z set
if_c_or_z	If either C or Z set
if_c_eq_z	If C equals Z

Condition	Meaning
<code>if_c_ne_z</code>	If C not equal to Z

And the comparison conditions (use after CMP):

Condition	Meaning
<code>if_a</code>	If above (unsigned greater)
<code>if_b</code>	If below (unsigned less)
<code>if_ae</code>	If above or equal
<code>if_be</code>	If below or equal

The Call/Return Dance

```

1      CALL    #subroutine    ' Call subroutine
2      RET                                ' Return from subroutine
3
4  ' But here's the P2 twist - CALL uses internal stack
5  subroutine
6      ' Do something useful
7      RET                                ' Returns to instruction after CALL
8
9  ' You get 8 levels of hardware stack!
```

The `_RET_` Prefix: Return With Benefits

Here's a clever trick the P2 offers. What if you could execute an instruction *and* return from a subroutine in one go? That's exactly what the `_RET_` prefix does.

```

1  ' Normal way: Two instructions
2  add_and_return
3      ADD      x, y            ' Do the add
4      RET                                ' Then return (4+ cycles total)
5  ' _RET_ way: One instruction!
6  add_and_return
7      _ret_    ADD      x, y    ' Add AND return (saves 2 cycles)
```

The `_RET_` prefix says: “Execute this instruction, then return.” It’s like getting a free return ticket with your instruction. The add happens, flags get set normally, and then—pop!—you’re back at the caller.

When does `_RET_` NOT return?

Here’s the catch: if the instruction itself branches, no return happens. The branch wins:

```

1      _ret_   JMP      #somewhere      ' JMP wins - no return
2      _ret_   CALL     #helper          ' CALL wins - no return
3      _ret_   DJNZ     count, #loop     ' Branch? No return. Zero? Return!
```

That last one is interesting! If `count` isn’t zero, `DJNZ` branches and no return. But when `count` hits zero, no branch occurs, so you get your return. Clever, right?

One-Instruction Subroutines

This is where `_RET_` really shines:

```

1  ' Toggle pin 0 - entire subroutine is ONE instruction!
2  toggle_led
3      _ret_   DRVNOT   #0                ' Toggle and return
4  ' Read all inputs - also just one instruction
5  read_inputs
6      _ret_   MOV      result, ina       ' Copy INA and return
7  ' Usage:
8      CALL    #toggle_led               ' Blink!
9      CALL    #read_inputs              ' result now has INA
```

A normal subroutine needs at least two instructions (the work + `RET`). With `_RET_`, you can have genuinely single-instruction subroutines. Your code gets smaller and faster.

The Medicine: `_RET_` Quick Reference

Pattern	What Happens
<code>_ret_ ADD x, y</code>	<code>ADD</code> executes, then return
<code>_ret_ JMP #label</code>	<code>JMP</code> executes, NO return (branch wins)
<code>_ret_ DJNZ n, #loop</code>	If <code>n>0</code> : branch, no return. If <code>n=0</code> : no branch, return

Important: Unlike `RET wcz`, the `_RET_` prefix does NOT restore C and Z flags from the stack. If you need flag restoration, use the regular `RET wcz` instruction.

Labels: Naming Your Places

You’ve been using labels throughout this chapter without us properly introducing them. How rude of me! Let’s fix that.

Global Labels: The Big Signposts

A global label is just a name at the start of a line:

```

1  DAT                ORG
2  send_byte          RDBYTE  x, ptr          ' Global label
3                      WYPIN   x, tx_pin
4                      RET
5  receive_byte        TESTP   rx_pin   wc    ' Another global label
6                      RDPIN   x, rx_pin
7                      RET

```

Global labels are visible everywhere in your DAT block. You can jump to them, call them, reference them from Spin2 - they’re your main signposts.

Local Labels: The Little Helpers

But here’s a problem. What if every routine needs a loop? You can’t have two labels called `loop` - the assembler would be terribly confused.

Enter local labels. Prefix a name with a dot (`.`) and it becomes local:

```

1  DAT                ORG
2  send_byte          RDBYTE  x, ptr
3  .loop              TESTP   tx_pin   wc    ' Local to send_byte
4      if_nc          JMP     #.loop
5                      WYPIN   x, tx_pin
6                      RET
7  receive_byte        TESTP   rx_pin   wc    ' New scope begins here
8      if_nc          JMP     #.wait
9  .wait              TESTP   rx_pin   wc    ' Local to receive_byte
10     if_nc          JMP     #.wait
11                      RDPIN   x, rx_pin
12  .loop              SHR     x, #24        ' Different .loop, OK!
13                      RET

```

Each global label starts a new “scope”. The `.loop` under `send_byte` is completely separate from the `.loop` under `receive_byte`. You can reuse `.loop`, `.done`, `.retry`, `.exit` to your heart’s content.

The Colon Alternative

You might also see local labels with a colon prefix:

```
1  :loop          DJNZ    count, #:loop    ' Same as .loop
```

Both `:` and `.` work identically. I prefer the dot - it’s what modern convention has settled on - but you’ll see both in the wild.

Reference Operators: Finding Your Labels

When you reference a label, you need to tell the assembler what you want:

```
1  ' In COG code (after ORG):
2      JMP      #my_routine    ' # = immediate COG address
3      CALL     #.helper       ' # works for local labels too
4      MOV      x, #data_table ' Get COG address of data
5  ' For hub addresses (used with Spin2):
6      MOV      ptr, @hub_data ' @ = hub address of label
```

The `#` means “immediate value” - use this for jumps and calls within COG code. The `@` means “hub address” - use this when passing addresses to Spin2 or for hub memory operations.

Scope Boundaries: When Local Labels Reset

Here’s the rule: **every global label or data definition starts a new local scope.**

```
1  func_a          MOV      x, #1          ' Scope #1 begins
2  .loop           DJNZ     x, #.loop       ' .loop in scope #1
3  data_block      LONG     0, 0, 0, 0     ' Scope #2 begins (data!)
4  func_b          MOV      y, #2          ' Scope #3 begins
5  .loop           DJNZ     y, #.loop       ' .loop in scope #3, OK!
6  .done           RET
```

This is wonderfully useful - your utility routines can all use `.loop` and `.done` without stepping on each other’s toes.

The Medicine: Quick Reference

What	Syntax	Example
Global label	<code>name</code>	<code>my_routine</code>
Local label	<code>.name</code> OR <code>:name</code>	<code>.loop, :done</code>
Jump to label	<code>#label</code>	<code>JMP #.loop</code>
Hub address	<code>@label</code>	<code>MOV ptr, @data</code>

Common Gotchas

1. **Forgetting the dot:** `loop` is global, `.loop` is local. If you accidentally create a global `loop`, you'll get conflicts.
2. **Scope surprise:** Data definitions (`LONG`, `WORD`, `BYTE`) also start new scopes. If you put data between two parts of a routine, your local labels won't work!
3. **The 30-character limit:** For compatibility with all tools, keep label names under 30 characters. `this_is_a_really_long_label_name` might cause trouble.

Data in DAT Blocks: Your Program's Pantry

Speaking of data definitions starting new scopes... we should probably talk about how to actually declare data! You've seen snippets like `counter LONG 0` scattered through our examples, but there's a whole world of data declaration waiting for you.

The Three Sizes

Just like Goldilocks, you have three choices:

```

1 DAT
2 my_byte      BYTE    $FF          ' 1 byte (8 bits)
3 my_word      WORD    $1234        ' 2 bytes (16 bits)
4 my_long      LONG    $DEADBEEF    ' 4 bytes (32 bits)
```

When do you use each? Well:

- **BYTE** for characters, small counters, flags, or when every byte of memory counts
- **WORD** for medium values, 16-bit peripherals, or when **BYTE** is too small but **LONG** is wasteful
- **LONG** for everything else - addresses, large numbers, and “I don't want to think about it”

If you're unsure, use LONG. Memory is cheap, debugging overflow errors is not.

Multiple Values on One Line

Here's a convenience - you can list multiple values after a single type:

```

1 DAT
2 primes          LONG    2, 3, 5, 7, 11, 13, 17, 19
3 gpio_pins       BYTE    16, 17, 18, 19, 20, 21
4 message         BYTE    "Hello, P2!", 0      ' String with null term

```

The assembler just lays them out consecutively in memory. That string? It's just bytes - each character followed by a zero at the end.

Arrays: The Repetition Trick

Need 100 bytes of zeros? Don't type them all out:

```

1 DAT
2 buffer          BYTE    0[100]              ' 100 zero bytes
3 lookup_table    WORD    $FFFF[64]           ' 64 words, all $FFFF
4 scratch_pad     LONG    0[32]               ' 32 longs of zeros

```

The [count] syntax repeats the value. This is your friend for buffers, tables, and anywhere you need initialized storage.

You can even combine values and repetition:

```

1 DAT
2 mixed_init      BYTE    $AA, $BB, 0[10], $CC, $DD
3                '      ^   ^   ^^^^^   ^   ^
4                '      Two vals, then 10 zeros, then two more

```

BYTEFIT and WORDFIT: The Safety Net

Here's a subtle trap. What if you accidentally write:

```

1 DAT
2 oops            BYTE    1000                ' 1000 won't fit in a byte!

```

The assembler will silently truncate 1000 to 232 (the low 8 bits). Your program will run, but with wrong values. Debugging that is no fun at all.

Enter the safety net:

```

1 DAT
2 safe_byte      bytefit 100, 200, 255    ' OK - all fit in a byte
3 danger_byte    bytefit 100, 200, 300    ' ERROR! 300 > 255, error!
4 safe_word      wordfit 1000, 50000      ' OK - all fit in a word
5 danger_word    wordfit 1000, 70000      ' ERROR! 70000 > 65535

```

Use `BYTEFIT` and `WORDFIT` when you want the assembler to check that your constants actually fit. It's like having a compiler catch your mistakes before they become 3 AM debugging sessions.

Alignment: Keeping Things Tidy

The P2 is quite forgiving about alignment - it can read a `LONG` from any address. But aligned access is faster and cleaner. Sometimes you need to force alignment:

```

1 DAT
2 some_bytes     BYTE    1, 2, 3          ' 3 bytes
3               ALIGNW                    ' Align to word boundary
4 next_word      WORD    $1234            ' Now properly aligned
5 more_bytes     BYTE    "ABC"            ' 3 bytes
6               ALIGNL                    ' Align to long boundary
7 next_long      LONG    $DEADBEEF        ' Now on a 4-byte boundary

```

When does alignment matter? Mostly when you're:

- Mixing data sizes in the same DAT block
- Creating structures that Spin2 code will access
- Optimizing for maximum hub access speed

If you're just starting out, don't worry about it. Add alignment when you hit problems.

A Complete Example

Let's put it all together:

```

1  DAT                ORG
2  ' Your code goes here
3  entry              MOV    ptra, ##buffer_addr
4                      MOV    count, #BUFFER_SIZE
5  .fill              WRBYTE  fill_value, ptra++
6                      DJNZ   count, #.fill
7                      JMP    #$
8  ' Constants (read-only, effectively)
9  fill_value         BYTE   $55
10 BUFFER_SIZE        LONG   256
11 ' Lookup tables
12                      ALIGNL
13 sin_table          WORD    0, 1608, 3212, 4808, 6393    ' Sine table
14                      WORD    7962, 9512, 11039, 12540    ' ... continues
15 ' Working storage
16                      ALIGNL
17 buffer_addr        LONG    0                          ' Set by Spin2 at startup
18 temp               LONG    0
19 result             LONG    0
20 ' Reserve uninitialized space
21                      ALIGNL
22 scratch            RES     16                          ' Reserve 16 longs

```

Notice the pattern: code first, then constants, then working storage, then reserved space. This keeps things organized and makes your DAT block readable.

The Medicine: Quick Reference

Declaration	Meaning	Example
BYTE val	8-bit value	BYTE \$FF
WORD val	16-bit value	WORD \$1234
LONG val	32-bit value	LONG \$DEADBEEF
val[n]	Repeat n times	BYTE 0[100]
bytefit	Byte with range check	bytefit 100, 200
wordfit	Word with range check	wordfit 1000, 50000
ALIGNW	Align to word	(no value)
ALIGNL	Align to long	(no value)

Declaration	Meaning	Example
<code>RES n</code>	Reserve n longs	<code>RES 16</code>

Common Gotchas

1. **Forgetting the label:** Every piece of data needs a label if you want to reference it. Anonymous data just wastes space.
2. **String termination:** `BYTE "Hello"` doesn't include a null terminator. Add `, 0` if you need one!
3. **RES is in longs:** `RES 10` reserves 10 *longs* (40 bytes), not 10 bytes. This trips up everyone at least once.
4. **Alignment after RES:** `RES` doesn't affect alignment. If you need alignment after reserved space, add an explicit `ALIGNL` or `ALIGNW`.

Including External Files: `FILE`

Sometimes you have binary data sitting in a file - a font bitmap, a sound sample, a pre-computed lookup table. Rather than manually converting it to hex values (ugh!), you can pull it straight in:

```

1 DAT
2 font_data      file    "myfont.bin"      ' Import entire file
3 sound_sample   file    "beep.raw"        ' Raw audio data
4 lut_table      file    "precalc.dat"     ' Pre-computed values
```

The assembler reads the file at compile time and drops its raw bytes right into your DAT block. The label gives you a way to reference where the data starts.

Where does it search?

1. Same folder as your source file
2. Library paths (if configured)

Practical example - embedded bitmap:

```

1 DAT                ORG
2 entry             MOV    ptr, ##@splash_screen
3                   ' ... display routine using ptr
4 ' The image data lives right here in your code
```

```
5 splash_screen    file    "logo_128x64.bin"    ' 1024 bytes of pixel data
```

No conversion scripts, no copy-paste errors. Just reference the file and it's part of your program. The icing on the cake? If you update the file, recompile, and your program has the new data.

String and Data Generation Methods

Beyond manually typing values, you have some helpers for creating data:

@“text” - Inline String Address

Need a string address without declaring a separate label?

```
1      MOV    ptra, @"Hello!"      ' ptra points to "Hello!" in hub
2      CALL   #print_string
3      MOV    ptra, @"Error: "     ' Another string, inline
4      CALL   #print_string
```

The `@“text”` syntax creates the string in hub memory and gives you its address. It's like an anonymous label for a string constant. Each unique string gets stored once, even if you reference it multiple times.

STRING(“text”) and LSTRING(“text”)

These work similarly but in different contexts:

```
1 ' In Spin2 code (not PASM):
2 DEBUG(STRING("Temperature: "))    ' Zero-terminated string address
3 DEBUG(LSTRING("Status"))          ' Length byte first, then string
```

`STRING()` returns the hub address of a zero-terminated string - same as what C programmers expect. `LSTRING()` puts a length byte at the front, which is handy when you need to know the string length without scanning for null.

BYTE[], WORD[], LONG[] - Data Arrays

In Spin2, you can create inline data arrays:

```
1 ' Spin2 examples:
2 lookup := BYTE[10, 20, 30, 40, 50]    ' Returns address of byte array
3 config := LONG[$DEAD_BEEF, $CAFE_BABE]
```


These are primarily Spin2 features, but they generate hub data that your PASM code can access if you know the addresses.

The Pattern:

Method	Result	Use Case
<code>file "name"</code>	Raw binary data	Images, audio, lookup tables
<code>@"text"</code>	String address	Quick inline strings in PASM
<code>STRING("text")</code>	Zero-terminated string address	Spin2 string constants
<code>LSTRING("text")</code>	Length-prefixed string	When you need length upfront

The Flags: C and Z (and Q!)

Flags are your friends. They remember things:

```

1  ' Z flag - was the result zero?
2      SUB    x, y wz      ' Set Z if x-y equals zero
3  if_z      JMP    #equal  ' Jump if they were equal
4  ' C flag - did we carry/borrow?
5      ADD    x, y wc      ' Set C if addition overflowed
6  if_c      JMP    #overflow ' Handle overflow
7  ' Both at once!
8      CMP    x, y wcz     ' Compare and set both flags
9  if_a      JMP    #x_greater ' Jump if x > y (unsigned)

```

The Q flag is special - it's used by CORDIC operations (Chapter 7).

Special Instructions That Will Blow Your Mind

SKIP - The Instruction Skipper

```

1      SKIP    ##%11010000  ' Skip pattern (1=skip, 0=execute)
2      ADD     x, #1         ' Skipped!
3      ADD     y, #1         ' Skipped!
4      ADD     z, #1         ' Executed
5      SUB     a, #1         ' Skipped!
6      SUB     b, #1         ' Executed

```

```
7      ' ... pattern continues
```

This is like having conditional execution on steroids!

REP - Hardware Loops

```
1      REP      #4, #5      ' Repeat next 4 instructions 5 times
2      ADD      x, #1
3      SUB      y, #1
4      ROL      z, #1
5      ROR      w, #1
6      ' These 4 instructions execute 5 times total
7      ' No loop overhead!
```

ALTD/ALTS - Instruction Modification

```
1  ' Modify the next instruction's destination
2      MOV      index, #10
3      ALTD     index, #array  ' Next instruction's dest = array+10
4      MOV      0-0, value     ' Actually moves to array[10]!
```

This replaces self-modifying code from P1. Much cleaner!

Real-World Example: Fast Memory Copy

Let's combine what we've learned:

```
1  ' Fast block copy using REP
2  fast_copy
3      MOV      ptra, ##source_addr  ' Source pointer
4      MOV      ptrb, ##dest_addr    ' Destination pointer
5
6      REP      #2, ##256            ' Repeat 256 times
7      RDLONG   temp, ptra++          ' Read and increment
8      WRLONG   temp, ptrb++          ' Write and increment
9      ' 1024 bytes (256 longs) copied with no loop overhead!
10
11 temp    LONG    0
```

The Medicine Cabinet

Feeling overwhelmed? Here's your simplified prescription:

Minimum Instructions to Know

```

1  ' Moving data
2      MOV      dest, source  ' Copy data
3  ' Math
4      ADD      dest, source  ' Addition
5      SUB      dest, source  ' Subtraction
6  ' Logic
7      AND      dest, source  ' AND operation
8      OR       dest, source  ' OR operation
9  ' Flow
10     JMP      #label        ' Jump
11     CALL     #label        ' Call subroutine
12     RET                      ' Return
13  ' Flags
14     CMP      x, y wcz      ' Compare and set flags
15     if_z     JMP      #label  ' Conditional jump

```

Master these 10 instructions and you can write real programs!

Common Gotchas

1. Immediate values:

- #value for 9-bit immediates (0-511)
- ##value for 32-bit immediates
- Forgetting # uses the register at that address!

2. Flag confusion:

- wz sets Z flag, wc sets C flag, wcz sets both
- No flag update means flags unchanged

3. PTR A/PTR B are special:

```

1  RDLONG  x, ptrA++        ' Read and auto-increment
2  RDLONG  x, ++ptrA        ' Pre-increment then read
3  RDLONG  x, ptrA--        ' Read and auto-decrement
4  RDLONG  x, ptrA[5]       ' Read from ptrA + 5*4

```

4. Address confusion:

- COG addresses are in longs (0-511)
- Hub addresses are in bytes (0-524287)

Your Turn: Experiments

Experiment 1: Conditional Counter

Count up if button pressed, down if not:

```

1          ORG      0
2
3  .loop    TESTP    #BUTTON_PIN wc ' Test button
4  if_c     ADD      counter, #1      ' Increment if pressed
5  if_nc    SUB      counter, #1      ' Decrement if not
6          WRLONG    counter, ##HUB_ADDR ' Display count
7          WAITX     ##1_000_000
8          JMP       #.loop
9
10 counter LONG      0

```

Experiment 2: Pattern Matcher

Find a pattern in data:

```

1          ORG      0
2
3          MOV       pattern, ##$DEADBEEF
4          MOV       ptr, ##data_start
5
6  .search  RDLONG    value, ptr++
7          CMP       value, pattern wz
8  if_z     JMP       #.found
9          CMP       ptr, ##data_end wcz
10 if_b     JMP       #.search
11          JMP       #.not_found
12 .found   ' Pattern found!
13          DRVH      #SUCCESS_LED
14          JMP       #$.

```

```
15 .not_found
16         DRVH    #FAIL_LED
17         JMP     #$_
```

Experiment 3: Speed Test

Compare multiply methods:

```
1  ' Method 1: Hardware multiply
2      GETCT    start_time
3      MUL      x, y
4      GETCT    end_time
5      SUB      end_time, start_time
6      ' Result: 2 clocks!
7
8  ' Method 2: Shift and add (old school)
9      GETCT    start_time
10     ' ... shift/add loop here
11     GETCT    end_time
12     ' Result: Many more clocks!
```

Sidetrack

Why PASM2 Is Different

Most assembly languages are thin wrappers over hardware. PASM2 is different - it's designed for humans:

1. **Symmetry:** Every instruction can use every addressing mode
2. **Orthogonality:** Features combine predictably
3. **Conditional everything:** Not just jumps, ANY instruction
4. **No special cases:** General-purpose registers, no accumulator

This isn't accident - it's philosophy. The P2 was designed to make assembly programming pleasant.

What We've Learned

- Instruction anatomy and structure
- Basic data movement and math
- Hardware multiply and divide (!)
- Conditional execution on any instruction
- Special instructions (SKIP, REP, ALT*)
- Flag operations and testing
- Why PASM2 is human-friendly

Coming Up Next

Chapter 4, “The Hub Connection”, explores:

- Reading and writing hub memory
- The FIFO and fast block transfers
- Hub execution mode
- Sharing data between COGs

You now speak basic PASM2. Time to learn how COGs communicate!

Have Fun! Remember, PASM2 isn't like other assembly languages - it's actually enjoyable!

Continue to Chapter 4: The Hub Connection →

Chapter 4: The Hub Connection

How COGs share and care

The Hook: Instant Communication

```

1  ' COG 1: Leave a message
2      WRLONG  ##$DEADBEEF, ##$1000
3
4  ' COG 2: Get the message
5      RDLONG  message, ##$1000
6      ' message now contains $DEADBEEF!
```

That's it - COGs talking through hub memory. But there's so much more...

Reading from Hub

The basics are simple:

```

1      RDBYTE  value, hubaddr    ' Read 1 byte
2      RDWORD  value, hubaddr    ' Read 2 bytes (word)
3      RDLONG  value, hubaddr    ' Read 4 bytes (long)
4
5  ' With PTRB magic
6      RDLONG  value, ptr++      ' Read and increment pointer
7      RDLONG  value, ++ptr      ' Increment then read
8      RDLONG  value, ptr[5]     ' Read from ptr + 5*4
```

Writing to Hub

Just as easy:

```

1      WRBYTE  value, hubaddr    ' Write 1 byte
2      WRWORD  value, hubaddr    ' Write 2 bytes
3      WRLONG  value, hubaddr    ' Write 4 bytes
4
5  ' The mighty block transfer
```

```
6          SETQ    #16-1          ' Transfer 16 longs
7          RDLONG  buffer, hubaddr ' Reads 16 longs in one go!
```

The FIFO Pipeline

Here's where P2 gets serious about speed:

```
1 ' Start the FIFO
2     RDFAST  #0, ##data_start ' Start fast read
3
4 ' Now read at maximum speed
5 .loop  RFLONG  value          ' Read from FIFO
6     ' Process value
7     DJNZ   count, #.loop      ' Decrement and jump if not zero
8
9 ' No hub timing worries - FIFO handles it all!
```


Real-World Example: Video Buffer

```
1  ' Fast screen clear using block transfer
2  ' Note: SETQ/SETQ2 maximum is 511 (for 512 longs)
3  ' For larger fills, we loop in 512-long chunks
4  clear_screen
5      MOV      hub_ptr, ##screen_buffer
6      MOV      chunks, ##640*480/512    ' Number of 512-long chunks
7      MOV      color, ##$00_00_00_00    ' Black
8  .loop  SETQ    #512-1                  ' Transfer 512 longs (max)
9          WRLONG color, hub_ptr          ' Fill this chunk
10         ADD    hub_ptr, ##512*4        ' Advance 512 longs (2KB)
11         DJNZ   chunks, #.loop
12         ' Full screen cleared with minimal loop overhead!
```

The Medicine Cabinet

Simple hub access pattern:

```
1  ' Just use PTR for everything
2      MOV     ptr, ##hub_address
3      RDLONG  value, ptr++
4      ' That's all you really need!
```

Continue to Chapter 5: Mathematics Unleashed →

Chapter 5: Mathematics Unleashed

Hardware multiply and divide - finally!

The Hook: Hardware Multiply

```

1      MUL      x, y              ' 16x16->32 bit unsigned multiply
2      ' Result in x (lower 16 bits of each operand used)
3      ' For full 32x32->64 bit multiply, use CORDIC:
4      QMUL     x, y              ' Start 32x32->64 multiply
5      ' ... other work (55 clocks) ...
6      GETQX    low               ' Get lower 32 bits
7      GETQY    high             ' Get upper 32 bits

```

Remember doing this with shifts and adds? Those days are over!

The Multiplication Revolution

```

1  ' Unsigned multiply
2      MUL      result, value     ' result = low 32 bits
3
4  ' Signed multiply
5      Muls     result, value     ' Signed version
6
7  ' Scale and multiply
8      SCL      result, ##$8000   ' Scale by 0.5 (32.32 fixed pt)

```

Division Without Tears

```

1  ' Start division
2      QDIV     dividend, divisor ' Start the operation
3
4  ' Get results (takes 30 clocks)
5      GETQX    quotient          ' Get quotient
6      GETQY    remainder         ' Get remainder
7

```

```

8 ' Fractional division
9     QFRAC    numerator, denominator
10    GETQX    fraction           ' 32-bit fraction

```

64-Bit Operations

```

1 ' 64-bit add
2     ADD      low1, low2 wc
3     ADDX     high1, high2
4 ' 64-bit multiply (uses CORDIC)
5     QMUL     x, y              ' Start 32x32->64 multiply
6     ' ... 55 clocks ...
7     GETQX    low               ' Lower 32 bits
8     GETQY    high              ' Upper 32 bits

```

Real-World Example: Fixed-Point Math

```

1 ' 16.16 fixed point multiply (uses CORDIC for full precision)
2 fixed_mul
3     QMUL     a, b              ' Start 32x32->64 signed multiply
4     ' ... 55 clocks (do other work) ...
5     GETQX    low               ' Lower 32 bits
6     GETQY    high              ' Upper 32 bits
7     ' Extract middle 32 bits for 16.16 result:
8     SHL      high, #16         ' Upper 16 bits of result
9     SHR      low, #16          ' Lower 16 bits of result
10    OR       a, low, high      ' Combine for 16.16 format

```

Continue to Chapter 6: Flags and Decisions →

Chapter 6: Flags and Decisions

Making choices at machine speed

The Hook: Any Instruction Can Be Conditional

```

1      CMP      x, y wcz      ' Compare x and y
2 if_a  MOV      result, x      ' If x > y, result = x
3 if_be  MOV      result, y      ' If x <= y, result = y
4      ' Max function in 3 instructions!
```

The C and Z Flags

```

1 ' Z Flag - Zero detection
2      SUB      x, y wz      ' Z=1 if x equals y
3 if_z  JMP      #equal      ' Jump if equal
4 ' C Flag - Carry/Borrow
5      ADD      x, y wc      ' C=1 if overflow
6 if_c  JMP      #overflow    ' Handle overflow
```

Complex Conditions

```

1 ' Combining flags
2      CMP      x, y wcz
3 if_a  JMP      #greater      ' x > y (unsigned)
4 if_b  JMP      #less         ' x < y (unsigned)
5 if_z  JMP      #equal        ' x == y
6 ' Signed comparisons
7      CMPS     x, y wcz      ' Signed compare
8 if_gt  JMP      #greater      ' x > y (signed)
9 if_lt  JMP      #less         ' x < y (signed)
```

Skip Patterns - Conditional Blocks

```
1      SKIPF    pattern          ' Set skip pattern
2      ADD     x, #1             ' Maybe executed
3      SUB     y, #1             ' Maybe executed
4      MOV     z, #0             ' Maybe executed
5      ' Pattern determines what runs!
```

Continue to Chapter 7: CORDIC Magic →

Chapter 7: CORDIC Magic

Trigonometry at the speed of logic gates

The Hook: Rotate a Point in 3 Lines

Here's something that would take dozens of instructions on most processors:

```

1  ' Rotate point (x,y) by angle - that's it!
2      SETQ    y_coord      ' Set Y coordinate
3      QROTATE x_coord, angle ' Start rotation by angle
4      GETQX   new_x        ' Get rotated X (55 clocks later)
5      GETQY   new_y        ' Get rotated Y

```

Three instructions. Point rotated. No lookup tables, no approximations, no floating point. Just pure mathematical precision delivered by dedicated hardware.

Let me show you something even more impressive:

```

1  ' Calculate sine and cosine simultaneously
2      QROTATE ##$7FFF_FFFF, angle ' D=radius (max), S=angle
3      GETQX   cosine             ' cos(angle) in 2.30 fixed pt
4      GETQY   sine               ' sin(angle) in 2.30 fixed pt
5      ' Both trig functions in 55 clocks total!

```

What Just Happened?

CORDIC stands for COordinate Rotation DIGital Computer. It's a method invented in 1959 for calculating trigonometric functions using only shifts and adds - no multiplies needed! Each P2 COG has its own dedicated CORDIC unit built into the hardware.

Think of CORDIC as your mathematical co-processor that can:

- Rotate points around the origin
- Convert between rectangular and polar coordinates
- Calculate sine, cosine, tangent
- Compute square roots and magnitudes
- Find arctangent (angle between points)
- Even do logarithms and exponentials!

All of this in exactly 55 clock cycles. Every time. No variation.

The CORDIC Pipeline - Your Mathematical Assembly Line

Here's the beautiful part: CORDIC operations are pipelined. While one calculation is running, you can start another:

```

1  ' Generate sine wave samples rapid-fire
2      MOV      angle, #0
3      MOV      count, #256
4  generate
5      QROTATE ##$7FFF_FFFF, angle    ' D=radius, S=angle
6      ADD      angle, ##$0100_0000   ' Increment angle (no wait!)
7
8      ' Do other work while CORDIC calculates
9      ADD      sample_ptr, #4
10     SUB      count, #1
11
12     GETQY    sample                ' Get sine result
13     WRLONG   sample, sample_ptr    ' Store it
14
15     TJNZ     count, #generate       ' Test-jump-not-zero
16     ' Generated 256 samples with perfect overlap!

```

The pipeline means you're not really waiting 55 clocks - you're getting useful work done while CORDIC churns away in the background!

Core CORDIC Operations

QROTATE - The Rotation Engine

Here's a subtle detail: CORDIC operations work on 2D coordinates (X, Y), but the **QROTATE** instruction only takes one coordinate directly. The solution? **SETQ** loads the Y coordinate into the Q register, then **QROTATE** takes X from its first operand. It's a two-instruction dance that becomes second nature:

```

1  ' Basic rotation: rotate point (x,y) by angle
2      SETQ     y                        ' First: load Y into Q register
3      QROTATE x, angle                 ' Then: X from operand, Y from Q

```



```

4          GETQX    new_x          ' Result: X' = X*cos( ) - Y*sin( )
5          GETQY    new_y          ' Result: Y' = X*sin( ) + Y*cos( )

```

The angle format is special: it's a 32-bit unsigned value where:

- \$0000_0000 = 0 degrees
- \$4000_0000 = 90 degrees
- \$8000_0000 = 180 degrees
- \$C000_0000 = 270 degrees
- \$FFFF_FFFF = just under 360 degrees

This makes angle math incredibly easy - just use regular addition and subtraction!

QVECTOR - From Rectangular to Polar

```

1  ' Convert (x,y) to polar (radius, angle)
2          SETQ      y          ' Load Y coordinate
3          QVECTOR x, #0        ' Start conversion
4          GETQX      radius     ' sqrt(x2 + y2)
5          GETQY      angle      ' atan2(y, x)

```

Perfect for:

- Finding distances between points
- Converting joystick input to angle/magnitude
- Radar and sonar applications

The Power of 32-Bit Precision

CORDIC uses 32-bit precision throughout:

- Angles: 32 bits (0.0000084 degree resolution!)
- Coordinates: 32 bits signed
- Results: Full 32-bit or 64-bit when needed

Real-World Example: Spinning a Sprite

Let's rotate a sprite around its center:

```

1  ' Rotate sprite vertices around center
2  rotate_sprite
3      MOV      vertex_ptr, ##sprite_data
4      MOV      vertex_count, #4          ' 4 corners
5
6  next_vertex
7      RDLONG   x, vertex_ptr++          ' Get X coordinate
8      RDLONG   y, vertex_ptr++          ' Get Y coordinate
9
10     ' Center sprite at origin
11     SUB      x, center_x
12     SUB      y, center_y
13
14     ' Rotate by current angle
15     SETQ     y
16     QROTATE  x, rotation_angle
17
18     ' While waiting, we can prepare
19     MOV      temp_x, center_x
20     MOV      temp_y, center_y
21
22     ' Get rotated coordinates
23     GETQX    x
24     GETQY    y
25
26     ' Translate back to position
27     ADD      x, temp_x
28     ADD      y, temp_y
29
30     ' Store rotated vertex
31     WRLONG   x, vertex_ptr++
32     WRLONG   y, vertex_ptr++
33
34     DJNZ     vertex_count, #next_vertex
35

```

```

36      ' Increment rotation for animation
37      ADD      rotation_angle, ##$0100_0000  ' ~1.4 deg (1/256 rotation)

```

Your Turn: CORDIC Experiments

Your Turn: Create a circular motion pattern

Starting code:

```

1      ORG      0
2
3      MOV      angle, #0
4      MOV      radius, ##100      ' 100 pixel radius
5 .loop  QROTATE radius, angle      ' D=X (radius), S=angle
6      ' Add code to:
7      ' 1. Get X,Y coordinates
8      ' 2. Add screen center offset
9      ' 3. Draw pixel at that position
10     ' 4. Increment angle
11     ' 5. Loop back to .loop

```

Goal: Make a dot trace a perfect circle on screen Hint: After qrotate, use getqx/getqy to get coordinates Success Check: Smooth circular motion, no gaps

Your Turn: Distance calculator

Starting code:

```

1  ' Calculate distance between two points
2      MOV      x1, #10
3      MOV      y1, #20
4      MOV      x2, #40
5      MOV      y2, #60
6
7      ' Calculate differences
8      SUB      x2, x1      ' dx
9      SUB      y2, y1      ' dy
10
11     ' Your code here: use qvector to find distance

```

Goal: Calculate the distance between the two points Hint: qvector with Y in Q gives you radius (distance) Success Check: Distance should be 50 units

The Medicine Cabinet

Feeling overwhelmed by all this trigonometry? Here's your simplified prescription:

Too Complex? Just remember these three patterns:

Pattern 1: Get sine/cosine

```
1      QROTATE ##$7FFF_FFFF, angle      ' D=radius, S=angle
2      GETQX   cos_value
3      GETQY   sin_value
```

Pattern 2: Rotate a point

```
1      SETQ    y
2      QROTATE x, angle
3      GETQX   new_x
4      GETQY   new_y
```

Pattern 3: Get distance

```
1      SETQ    dy
2      QVECTOR dx, #0
3      GETQX   distance
```

Master these three and you can do 90% of what you need!

Advanced CORDIC: The Pipeline Dance

Here's where CORDIC gets really powerful - overlapping operations:

```
1  ' Process multiple points while calculating
2  process_points
3      MOV      count, #16
4      MOV      ptr, ##point_array
5
6      ' Start first calculation
7      RDLONG   x, ptr++
```

```

 8      RDLONG  y, ptra++
 9      SETQ    y
10      QROTATE x, angle
11
12 process_loop
13      ' Start next calculation immediately
14      RDLONG  x, ptra++ wz      ' Z flag tells us if done
15      if_nz RDLONG  y, ptra++
16      if_nz SETQ    y
17      if_nz QROTATE x, angle      ' New calculation starts
18
19      ' Get previous result
20      GETQX  prev_x
21      GETQY  prev_y
22
23      ' Store previous result
24      WRLONG prev_x, ptrb++
25      WRLONG prev_y, ptrb++
26
27      DJNZ   count, #process_loop
28
29      ' Don't forget last result!
30      GETQX  prev_x
31      GETQY  prev_y
32      WRLONG prev_x, ptrb++
33      WRLONG prev_y, ptrb++

```

See what happened? We started each new CORDIC operation immediately after the previous one, then retrieved results later. This pipeline approach means we're effectively getting one rotation every few instructions instead of waiting 55 clocks each time!

CORDIC for Graphics

Want to draw a spiral? CORDIC makes it trivial:

```

 1  ' Expanding spiral generator
 2  spiral
 3      MOV    angle, #0

```

```

4      MOV      radius, #1
5  draw_spiral
6      QROTATE  radius, angle      ' D=X (radius), S=angle
7      GETQX   x
8      GETQY   y
9
10     ' Convert to screen coordinates
11     SAR      x, #16              ' Scale down
12     SAR      y, #16
13     ADD      x, #320             ' Center X
14     ADD      y, #240             ' Center Y
15
16     ' Plot pixel (simplified)
17     CALL     #plot_pixel
18
19     ' Expand spiral
20     ADD      angle, ##$0400_0000 ' Rotate ~22.5 degrees
21     ADD      radius, ##100        ' Expand slowly
22
23     CMP      radius, ##30000 wcz
24     if_b JMP      #draw_spiral

```

CORDIC for Audio

Generate perfect sine waves for audio:

```

1  ' Audio tone generator using CORDIC
2  tone_generator
3      MOV      phase, #0
4      MOV      frequency, ##$0100_0000 ' ~1.4 deg/sample (1/256 rot)
5
6  sample_loop
7      QROTATE  ##$7FFF_FFFF, phase    ' D=radius, S=angle
8      ADD      phase, frequency       ' Increment phase
9
10     ' Do other audio processing while waiting
11     RDLONG   volume, ##volume_addr
12

```

```

13      GETQY    sample                ' Get sine value
14      SAR      sample, #16           ' Scale to 16-bit
15      MULS     sample, volume        ' Apply volume
16
17      ' Output to DAC
18      WYPIN    sample, #AUDIO_PIN
19
20      ' Wait for sample period (48kHz)
21      WAITX    ##4166                ' 200MHz / 48kHz
22
23      JMP      #sample_loop

```

Common CORDIC Gotchas

Before you pull your hair out debugging, know these:

1. **One result at a time** - Each COG has its own CORDIC, but starting a new operation before retrieving your result overwrites it!
2. **55 clocks is exact** - Not 54, not 56. Always exactly 55 clocks from operation start to result ready.
3. **Don't forget SETQ** - For two-operand operations (QROTATE with X,Y), you must load Y into Q first.
4. **Results are scaled** - When rotating by unit circle (\$7FFF_FFFF), results are in 2.30 fixed point format.
5. **Angles wrap naturally** - Adding \$1_0000_0000 to an angle is the same as adding 0. Use this!

What About QLOG, QEXP?

CORDIC can also do logarithms and exponentials:

```

1  ' Natural logarithm
2      QLOG     value
3      GETQX    result                ' ln(value) in 5.27 fixed point
4
5  ' Exponential
6      QEXP     value
7      GETQX    result                ' e^value

```

These are less commonly used but incredibly powerful for DSP and scientific calculations.

Interlude

Jack Volder's Gift to Computing

In 1959, Jack Volder was working on navigation computers for aircraft. He needed to calculate trigonometric functions, but the computers of the day couldn't handle the complex math quickly enough.

His insight? Any angle can be decomposed into a sequence of smaller, fixed angles. By choosing these angles cleverly (arctan of powers of 2), he could rotate vectors using only shifts and adds - no multiplication needed!

The B-58 bomber's navigation computer was the first to use CORDIC. Today, it's in your P2, calculating sines and cosines faster than those room-sized computers could add two numbers.

From military navigation to your LED projects - quite a journey for an algorithm!

What We've Learned

Let's celebrate your new CORDIC powers:

- Understood CORDIC's rotate and vector operations
- Generated sine and cosine values
- Calculated distances and angles
- Learned the pipeline technique for speed
- Created rotating graphics
- Built an audio tone generator

That's serious mathematical muscle!

Coming Up Next

Chapter 8 brings us back to Earth with "Basic I/O" - the fundamental pin operations that make the real world connection. We'll save Smart Pins for another manual and focus on the essentials: making pins go high and low, reading buttons, and basic timing.

But first, take a moment to appreciate what you just learned. CORDIC is unique to the Propeller 2 - most microcontrollers would need extensive software libraries to do what you just did in three instructions!

Have Fun! And remember - with CORDIC, you're not just calculating trigonometry, you're doing it at hardware speed. That's magical!

Chapter 8: Basic I/O

Making the real world connection

The Hook: One Pin, Three Instructions, Infinite Possibilities

Watch this:

```
1 ' Complete button-and-LED program
2 .loop  TESTP  #BUTTON_PIN wc  ' Read button into C flag
3     if_c  DRVH  #LED_PIN      ' If pressed, LED on
4     if_nc  DRVL  #LED_PIN      ' If not pressed, LED off
5         JMP   #.loop          ' Repeat forever
```

Four lines. Complete input/output program. No configuration registers, no data direction setup, no port manipulation. Just pure, simple I/O.

But wait, let me show you the same thing with even more elegance:

```
1 ' Even simpler - button controls LED directly
2 .loop  TESTP  #BUTTON_PIN wc  ' Read button
3         DRVC  #LED_PIN        ' Drive LED from C flag!
4         JMP   #.loop
```

Three lines! The DRVC instruction drives the pin to match the C flag. Input becomes output. Simple becomes simpler.

Understanding P2 Pins

Every P2 pin is bidirectional and incredibly capable. Unlike older microcontrollers where you set data direction registers, P2 pins change direction on the fly based on the instruction you use.

Here's the mental model:

- **Output instructions** automatically make the pin an output
- **Input instructions** automatically make the pin an input
- **Float instructions** make the pin high-impedance
- No setup required!

Digital Output: Making Things Happen

The Fundamental Four

```
1      DRVH    #56      ' Drive pin 56 HIGH (3.3V)
2      DRVL    #56      ' Drive pin 56 LOW (0V)
3      DRVNOT  #56      ' Toggle pin 56
4      FLTL    #56      ' Float pin 56 (high-Z)
```

That's it. These four instructions cover 90% of your output needs.

Conditional Driving

Here's where P2 gets clever:

```
1      DRVH    #56      ' Drive pin to match C flag
2      DRVNC    #56      ' Drive pin to NOT C flag
3      DRVZ     #56      ' Drive pin to match Z flag
4      DRVNZ    #56      ' Drive pin to NOT Z flag
```

And the really clever one:

```
1      DRVNOT  #56 wcz      ' Toggle pin AND read old state to C
2      ' C now contains what the pin WAS before toggling
```

Random and Pattern Outputs

```
1      drvrnot #56      ' Randomly toggle (hardware random!)
2      OUTL    #56      ' Drive low (alternate form)
3      OUTH    #56      ' Drive high (alternate form)
```

Digital Input: Reading the World

Basic Pin Reading

```
1      TESTP    #BUTTON_PIN wc ' Read pin into C flag
2      if_c JMP    #pressed      ' Branch if high
3      if_nc JMP    #not_pressed  ' Branch if low
```

Or read into Z flag for zero/non-zero testing:

```
1      TESTP    #SENSOR_PIN wz ' Read pin into Z flag
2      if_z JMP    #sensor_low   ' Jump if pin low (Z=1 when pin=0)
3      if_nz JMP    #sensor_high ' Jump if pin is high
```

Reading Multiple Pins

```
1 ' Read 8 pins at once (pins 0-7)
2      MOV      mask, #$FF      ' Pins 0-7
3      TESTB    ina, #0 wc      ' Test pin 0
4      RCL      result, #1      ' Rotate C into result
5      TESTB    ina, #1 wc      ' Test pin 1
6      RCL      result, #1
7      ' ... continue for all 8 pins
```

Pin Timing: When Things Happen

Waiting for Pin Changes

```
1 ' Wait for pin to go high
2 wait_high
3      TESTP    #SIGNAL_PIN wc
4      if_nc JMP    #wait_high
5
6 ' Wait for pin to go low
7 wait_low
8      TESTP    #SIGNAL_PIN wc
9      if_c JMP    #wait_low
```

But there's a better way - hardware-assisted waiting with Smart Events:

```

1      WAITSE1          ' Wait for event 1
2      WAITSE2          ' Wait for event 2
3      ' Configure events to watch pins - super efficient!
```

Real-World Example: Button Debouncing

Mechanical buttons bounce. Here's how to handle it:

```

1  ' Debounced button reader
2  read_button
3      MOV      debounce, #0
4
5  check_button
6      TESTP    #BUTTON_PIN wc
7      if_c ADD    debounce, #1      ' Count high readings
8      if_nc MOV    debounce, #0      ' Reset on any low
9
10     CMP      debounce, #10 wcz ' Need 10 consecutive highs
11     if_ae JMP    #button_confirmed
12
13     WAITX     ##200_000          ' Wait 1ms at 200MHz
14     JMP      #check_button
15
16 button_confirmed
17     ' Button definitely pressed
18     DRVH     #LED_PIN
```

Bit-Banged Serial (The Basics)

Sometimes you need serial communication without Smart Pins. Here's how:

```

1  ' Bit-bang serial transmit at 115200 baud
2  tx_byte
3      OR      data, ##$100      ' Add stop bit
4      SHL     data, #1          ' Add start bit (0)
5      MOV     bits, #10         ' 1 start + 8 data + 1 stop
```

```

6
7      GETCT    time          ' Get current time
8
9 tx_loop
10     SHR      data, #1 wc    ' Get next bit into C
11     DRVC     #TX_PIN        ' Output bit
12
13     ADDCT1    time, bit_time ' Next bit time
14     WAITCT1           ' Wait for it
15
16     DJNZ      bits, #tx_loop
17     RET
18
19 bit_time LONG 100_000_000 / 115200 ' Clock cycles per bit

```

Your Turn: I/O Experiments

Your Turn: Create a light chaser

Starting code:

```

1      ORG      0
2
3      MOV      pattern, #1    ' Start with one LED
4
5 .loop  MOV      pins, pattern  ' Your code here
6      ' Make pattern rotate through pins 56-63
7      ' Add delay between changes
8      ' Wrap around at the end, then jmp #.loop

```

Goal: Create a rotating light pattern on LEDs Hint: Use SHL and check for overflow Success Check: Single lit LED rotating through all positions

Your Turn: Reaction timer

Starting code:

```

1      ORG      0
2

```

```

3      ' Turn on LED after random delay
4      GETRND  delay
5      AND     delay, ##$3FFF_FFFF  ' Limit range
6      WAITX   delay
7      DRVH    #LED_PIN
8
9      GETCT   start_time
10     ' Your code: wait for button press
11     ' Calculate reaction time

```

Goal: Measure reaction time between LED and button press Hint: Use `getc` after button detection
 Success Check: Time measured in clock cycles

The Medicine Cabinet

Feeling overwhelmed by all these pin operations? Here's the simplified prescription:

Just need something working? Remember these patterns:

Output pattern:

```

1      DRVH    #PIN    ' Make it high
2      DRVL    #PIN    ' Make it low
3      DRVNOT  #PIN    ' Toggle it

```

Input pattern:

```

1      TESTP   #PIN wc ' Read it
2      if_c    JMP    #high ' It's high
3      if_nc    JMP    #low  ' It's low

```

Timed pattern:

```

1  .loop  DRVNOT  #LED
2          WAITX   ##50_000_000
3          JMP     #.loop

```

That's 80% of all I/O right there!

Advanced Pin Control

Pin Groups

You can control multiple pins at once:

```
1          DRVH    #LED_BASE addpins 3    ' Drive 4 pins high (base+3)
2          DRVL    #LED_BASE addpins 7    ' Drive 8 pins low
```

Direct Pin Manipulation

For when you need absolute control:

```
1          MOV      outa, pattern          ' Set output register directly
2          MOV      dira, ##$FF           ' Set direction reg (rare in P2!)
```

But honestly? You'll rarely need these. The individual pin instructions are cleaner and clearer.

Common I/O Gotchas

Save yourself debugging time:

1. **Pin numbers are 0-63** - Not port.bit notation like other MCUs
2. **No pullup/pulldown by default** - Use external resistors or configure Smart Pin modes (advanced topic)
3. **Pins float on reset** - All pins start as inputs (floating)
4. **Reading output pins** - You CAN read a pin you're driving (reads the actual pin state)
5. **3.3V logic levels** - P2 is 3.3V, not 5V tolerant!

Timing Is Everything

Here's a critical concept: P2 I/O is deterministic. When you execute:

```
1          DRVH    #56
2          DRVL    #57
```

Pin 56 goes high and pin 57 goes low at EXACTLY the same clock cycle. No skew, no uncertainty. This determinism is what makes P2 perfect for precise timing applications.

Real-World Example: Servo Control

Even without Smart Pins, controlling a servo is easy:

```
1  ' Standard servo control (1-2ms pulse every 20ms)
2  servo_control
3      MOV      position, ##300_000      ' 1.5ms = center at 200MHz
4  servo_loop
5      DRVH     #SERVO_PIN
6      WAITX    position                  ' 1-2ms high pulse
7      DRVL     #SERVO_PIN
8      WAITX    ##4_000_000              ' Rest of 20ms at 200MHz
9      ' Adjust position as needed
10     RDLONG   position, ##position_addr
11     FLE      position, ##200_000      ' Limit to 1ms min
12     FGE      position, ##400_000      ' Limit to 2ms max
13
14     JMP      #servo_loop
```

The Power of Simple

Here's something beautiful about P2's I/O philosophy: it's transparent. Unlike microcontrollers with complex GPIO configurations, port multiplexing, and alternate functions, P2 pins just... work.

Want an output? Drive it. Want an input? Read it. Want it to float? Float it.

No setup, no configuration, no confusion.

What We've Learned

Look at your new I/O skills:

- Understood P2's automatic pin direction
- Mastered the four fundamental output instructions
- Learned pin reading and conditional testing
- Created debounced inputs
- Built bit-banged serial
- Discovered deterministic timing

A Note About Smart Pins

You might wonder - if basic I/O is this simple, why do we need Smart Pins?

Well, while you CAN bit-bang serial at 115200 baud, or generate PWM, or measure frequencies using the techniques in this chapter, Smart Pins do all of this in hardware, freeing your COG for more important work.

For Smart Pin details: See the dedicated “P2 Smart Pins Manual” which covers all 64 modes, from simple PWM to complex protocol generation. Smart Pins deserve their own complete treatment!

Coming Up Next

Chapter 9 takes us into “Streaming Data” - the P2’s incredible FIFO system that can move megabytes of data without breaking a sweat. We’ll see how to stream video, audio, and massive data blocks at maximum speed.

Have Fun! Remember, every embedded system ultimately comes down to pins going high and low. You’ve just mastered the fundamentals that everything else builds upon!

Chapter 9: Streaming Data

Moving mountains of data without breaking a sweat

The Hook: 4KB in 4 Instructions

Watch this data transfer magic:

```
1 ' Copy 1000 longs (4KB) at maximum speed
2     SETQ    ##1000-1      ' Setup for 1000 longs
3     RDLONG  buffer, source ' Read them all!
4     SETQ    ##1000-1      ' Setup for 1000 longs
5     WRLONG  buffer, dest   ' Write them all!
6     ' 4KB moved in microseconds!
```

Four instructions. Four kilobytes. Faster than DMA on most processors. And we're just getting started...

Block Transfers: The Power Move

The SETQ instruction is your gateway to block transfers:

```
1 ' Basic block read
2     SETQ    #16-1          ' Transfer 16 longs (minus 1!)
3     RDLONG  buffer, hubaddr ' Reads 16 consecutive longs
4
5 ' Basic block write
6     SETQ    #16-1          ' Transfer 16 longs
7     WRLONG  buffer, hubaddr ' Writes 16 consecutive longs
```

The key insight: SETQ tells the next hub instruction how many longs to transfer. The “-1” is because it’s a count from 0.

The FIFO: Your Streaming Pipeline

The FIFO (First In, First Out) is P2’s streaming engine. Think of it as a conveyor belt between hub memory and your COG:

```

1  ' Start FIFO reading
2      RFAST  #0, ##data_start  ' Start fast read at data_start
3
4  ' Now read data at maximum speed
5  stream_loop
6      RFLONG  value              ' Read from FIFO (no waiting!)
7      ' Process value here
8      ADD     accumulator, value
9      DJNZ    count, #stream_loop
10
11 ' The FIFO keeps feeding data automatically

```

The beauty? The FIFO reads ahead automatically. While you're processing one value, it's already fetching the next. No hub timing slots to worry about!

Writing Through the FIFO

```

1  ' Start FIFO writing
2      WRFAST  #0, ##dest_buffer
3
4  ' Stream data out
5  write_loop
6      ' Generate or process data
7      MOV     value, calculation
8      WFLONG  value              ' Write to FIFO
9      DJNZ    count, #write_loop
10
11 ' Data streams to hub automatically

```

Real-World Example: Screen Buffer Clear

Let's clear a 320x240x4 byte screen buffer (~307KB - fits in hub!):

```

1  ' Ultra-fast screen clear
2  clear_screen
3      MOV     color, ##$00_00_00_00  ' Black (4 bytes per pixel)
4      MOV     pixels, ##320*240      ' 76,800 pixels

```

```

5          WRFAST  #0, ##screen_buffer      ' Start FIFO write
6  clear_loop
7          WFLONG  color                    ' Write 4-byte pixel
8          DJNZ    pixels, #clear_loop
9          ' 307KB cleared at maximum hub speed!
10         ' Note: Hub RAM is 512KB - plan buffer sizes accordingly

```

Streaming with the Streamer

The Streamer is different from the FIFO - it's a dedicated DMA engine that can move data between hub memory and pins:

```

1  ' Configure streamer for video output
2      SETCMOD  #$100          ' Set color mode
3      SETCY    ##640          ' Cycles per line
4      SETCI    ##LINE_TIME    ' Line timing
5
6  ' Start streaming video data to pins
7      XINIT    ##STREAM_CMD, #0 ' Start streamer
8      ' Data flows from hub to pins automatically!

```

FIFO and COG Execution

Here's something amazing - you can execute code from hub through the FIFO:

```

1  ' Execute large program from hub
2      ORGH     $1000          ' Code in hub memory
3
4  hub_code
5      ' This code is in hub but executes like it's in COG
6      ADD      x, y
7      SUB      a, b
8      ' Can be megabytes of code!

```

When you call or jump to hub code, the FIFO automatically feeds instructions to the COG. It's like having unlimited code space!

The Medicine Cabinet

Feeling overwhelmed by all this streaming? Here's your prescription:

Just need to move data? Use these simple patterns:

Block read pattern:

```
1      SETQ    #SIZE-1
2      RDLONG  buffer, source
```

Block write pattern:

```
1      SETQ    #SIZE-1
2      WRLONG  buffer, dest
```

FIFO read pattern:

```
1      RDFAST  #0, ##source
2  .loop  RFLONG  value
3          ' Process value
4      DJNZ    count, #.loop
```

That's 90% of streaming right there!

Advanced Streaming Techniques

Circular Buffers with FIFO

```
1  ' Circular buffer reading
2      RDFAST  ##$8000_0000, ##buffer  ' Bit 31 set = wrap mode
3
4  circular_loop
5      RFLONG  value                    ' Read from FIFO
6      ' Process value
7      ' FIFO automatically wraps at buffer end!
8      JMP     #circular_loop
```

Processing Pipeline with FIFO

```

1  ' Read data with FIFO, process, write via PTR
2      RDFAST  #0, ##source      ' Set up FIFO for reading
3      MOV     dest_ptr, ##dest
4  pipeline
5      RFLONG  input              ' Get next input from FIFO
6      ' Scale using 16x16 multiply (result in input)
7      MUL     input, #SCALE_FACTOR
8      WRLONG  input, dest_ptr ' Write result via PTR
9      ADD     dest_ptr, #4
10     DJNZ    count, #pipeline

```

Note: FIFO can only read OR write at a time, not both. Use PTRB/PTRB for the other direction.

Your Turn: Streaming Experiments

Your Turn: Fast memory fill

Starting code:

```

1      ORG     0
2
3      MOV     pattern, ##$DEADBEEF
4      MOV     dest, ##$1000
5      MOV     count, #256
6
7      ' Your code here: Fill 256 longs with pattern
8      ' Use SETQ and WRLONG

```

Goal: Fill memory with pattern using block transfer Hint: You'll need setq #255 (not #256)

Success Check: Memory filled in one operation

Your Turn: Data filter pipeline

Starting code:

```

1      ORG     0
2      RDFAST  #0, ##input_data      ' FIFO for reading
3      MOV     ptr, ##output_data    ' PTR for writing

```

```

4      MOV      count, #100
5  filter_loop
6      RFLONG   value                ' Read from FIFO
7      ' Your code: Simple filter
8      ' Maybe average with previous value?
9      WRLONG   result, ptr++        ' Write via PTR
10     DJNZ     count, #filter_loop

```

Goal: Process streaming data through simple filter Hint: Keep previous value in a register Success

Check: Output is filtered version of input

Common Streaming Gotchas

Watch out for these:

1. **SETQ uses count-1** - For 16 longs, use SETQ #15, not SETQ #16
2. **FIFO is shared per COG** - Can't use FIFO for both code execution and data streaming simultaneously
3. **Write synchronization** - WRFAST doesn't wait for writes to complete. Use WAITX #20 if you need to ensure completion
4. **Hub alignment** - Block transfers work best with long-aligned addresses
5. **FIFO depth** - The FIFO is 64 longs deep. Don't outrun it!

Performance Numbers

Let's talk speed:

- **Block transfer:** Up to 1 long per clock (at 200MHz = 800MB/s!)
- **FIFO streaming:** Up to 1 long per clock sustained
- **Random hub access:** 2-9 clocks per access
- **Streamer to pins:** Up to sysclock/1 rate

This is seriously fast. Most microcontrollers need dedicated DMA controllers to achieve what P2 does with simple instructions.

Real-World Example: Audio Buffer

Stream audio samples through processing:

```
1  ' Audio processing pipeline
2  audio_process
3      RDFAST  #0, ##input_buffer      ' FIFO for reading input
4      MOV     ptra, ##output_buffer   ' PTR for writing output
5      MOV     samples, ##BUFFER_SIZE
6  process_loop
7      RFLONG  left_sample              ' Read left from FIFO
8      RFLONG  right_sample             ' Read right from FIFO
9      ' Apply simple low-pass filter
10     ADD     left_filtered, left_sample
11     SHR     left_filtered, #1        ' Average with previous
12     ADD     right_filtered, right_sample
13     SHR     right_filtered, #1
14     ' Apply volume
15     Muls     left_filtered, volume
16     Muls     right_filtered, volume
17     ' Output processed samples via PTR
18     WRLONG  left_filtered, ptr++
19     WRLONG  right_filtered, ptr++
20     DJNZ    samples, #process_loop
```

What We've Learned

Your streaming skills now include:

- Block transfers with SETQ
- FIFO reading and writing
- Streaming pipeline concepts
- Circular buffer techniques
- Parallel processing while streaming
- Real-world applications

Coming Up Next

Chapter 10 explores “Hub Execution” - how to break free from the 512-instruction limit and run massive programs directly from hub memory. It’s like having your cake and eating it too!

Have Fun! Remember, streaming is about throughput, not just speed. It’s the difference between carrying one brick at a time and using a wheelbarrow!

Chapter 10: Hub Execution

Breaking free from the 512-instruction limit

The Hook: Unlimited Code Space

Remember fretting about fitting your code into 496 COG instructions? Watch this:

```
1      ORGH    $400          ' Place code in hub memory
2
3      ' This can be thousands of instructions!
4 main    MOV     x, #0
5          MOV     y, #0
6          CALL    #huge_function  ' Can be massive
7          CALL    #another_big_one
8          CALL    #yet_another
9          ' Keep going... no limit!
10
11 huge_function
12      ' 1000 instructions? No problem!
13      ' 10000 instructions? Still fine!
14      RET
```

Your code now lives in hub memory's 512KB instead of COG memory's 2KB. That's 256 times more space!

COG vs Hub Execution: The Trade-offs

Let's be honest about the differences:

COG Execution (traditional):

- Fast: exactly 2 clocks per instruction
- Deterministic: perfect for real-time
- Limited: only 496 instructions
- Self-contained: runs independently

Hub Execution (the new way):

- Slower: 2-9 clocks per instruction (typically 3-4)

- Variable timing: depends on hub alignment
- Unlimited: 512KB of code space!
- Flexible: can call COG routines

The beauty? You can mix both in the same program!

How Hub Execution Works

When the processor encounters a jump or call to a hub address (\$400), it automatically switches to hub execution mode. The FIFO starts streaming instructions from hub memory:

```
1      ORG      0              ' Start in COG
2
3  cog_code
4      ' This runs from COG RAM
5      CALL    #hub_function  ' Call into hub
6      ' Back in COG mode here
7
8      ORGH    $1000          ' Switch to hub addresses
9
10 hub_function
11      ' This runs from hub RAM via FIFO
12      ' Can be huge!
13      RET                ' Returns to COG code
```

The magic happens automatically. No mode switching instructions needed!

Real-World Example: Menu System

Here's something that would never fit in COG RAM:

```
1      ORGH    $2000
2
3  menu_system
4      CALL    #draw_menu_frame
5      CALL    #display_options
6      CALL    #get_user_input
7      CALL    #process_selection
8
```

```

 9      CMP      selection, #1 wcz
10  if_e CALL    #option_1_handler
11      CMP      selection, #2 wcz
12  if_e CALL    #option_2_handler
13      CMP      selection, #3 wcz
14  if_e CALL    #option_3_handler
15      ' ... many more options
16
17      JMP      #menu_system
18
19 draw_menu_frame
20      ' 200 instructions for fancy graphics
21      RET
22
23 display_options
24      ' 300 instructions for text rendering
25      RET
26
27 option_1_handler
28      ' 500 instructions for configuration
29      RET
30
31 ' Thousands of instructions total - no problem!

```

The Hub Execution FIFO

The FIFO that makes hub execution possible is the same one used for streaming data. It reads ahead, keeping a buffer of upcoming instructions:

```

1  ' The FIFO maintains performance by reading ahead
2  hub_loop
3      ADD      x, y          ' FIFO has next instructions ready
4      SUB      a, b          ' No waiting for hub access
5      MUL      c, d          ' Instructions stream smoothly
6      ' FIFO automatically refills as needed

```

This read-ahead behavior means hub execution is often faster than the worst-case 9 clocks per instruction.

Mixing COG and Hub Code

Here's the real power - combining both modes:

```

1          ORG      0
2
3  ' Critical timing code in COG
4  critical_loop
5          TESTP    #TRIGGER_PIN wz      ' Test trigger pin
6  if_nz JMP      #critical_loop        ' Loop until triggered
7          DRVH     #CRITICAL_PIN        ' Immediate response!
8          CALL     #hub_process         ' Do complex processing
9          JMP      #critical_loop
10
11         ORGH     $4000
12
13  ' Complex processing in hub
14  hub_process
15      ' Hundreds of instructions for data analysis
16      ' Not time-critical, so hub execution is fine
17      RET

```

Time-critical code stays in COG RAM for deterministic timing. Complex code lives in hub RAM for space.

Your Turn: Hub Execution Experiments

Your Turn: Build a simple calculator

Starting code:

```

1          ORG      0
2          JMP      #calculator         ' Jump to hub code
3
4          ORGH     $1000
5  calculator
6      ' Your code here:
7      ' 1. Display menu
8      ' 2. Get operation choice
9      ' 3. Get two numbers

```

```

10         ' 4. Call appropriate function
11         ' 5. Display result
12
13 add_function
14         ' Addition code
15         RET
16
17 subtract_function
18         ' Subtraction code
19         RET
20
21 ' Add more functions - no size limit!

```

Goal: Create a multi-function calculator Hint: Each function can be as complex as needed Success
 Check: Multiple operations working

The Medicine Cabinet

Overwhelmed by execution modes? Here's the simple version:

Keep it simple:

1. **Small, time-critical code** → Put in COG (org 0)
2. **Large, complex code** → Put in hub (orgh \$400+)
3. **Don't overthink it** → The processor handles the switch

Basic pattern:

```

1         ORG      0
2         JMP      #main      ' Jump to hub
3         ORGH     $400
4 main    ' Your big program here

```

That's it. Let the processor worry about the details!

Advanced Hub Execution

Long Jumps and Calls

Hub addresses need 20 bits, so jumping far requires special handling:

```

1 ' Jump to distant hub code
2     JMP     ##far_away      ' ## forces 32-bit immediate
3
4     ORGH    $40000          ' Far away in hub
5 far_away
6     ' Code here

```

Hub Data Access from Hub Code

When executing from hub, you can still access hub data:

```

1     ORGH    $1000
2
3 hub_code
4     RDLONG  value, ##hub_data ' Read hub data
5     ADD     value, #1
6     WRLONG  value, ##hub_data ' Write back
7
8     ORGH    $8000
9 hub_data
10    LONG    $12345678

```

Performance Optimization

To maximize hub execution speed:

```

1 ' Align branch targets to 8-byte boundaries
2     ALIGNL                      ' Align to long boundary
3 loop_start
4     ' Loop code here
5     DJNZ    count, #loop_start
6
7 ' Keep critical loops small
8 ' Consider moving inner loops to COG RAM

```

Common Hub Execution Gotchas

1. **Speed variation** - Don't use hub execution for precise timing
2. **FIFO conflicts** - Can't stream data while executing from hub
3. **Address confusion** - Remember: <\$200 is COG, \$200-\$3FF is LUT, \$400 is hub
4. **Stack depth** - Still limited to 8-level hardware stack
5. **Relative jumps** - Work differently in hub mode

Real-World Example: Command Parser

```
1          ORGH    $2000
2
3  command_parser
4          CALL    #get_command_line
5          CALL    #tokenize
6
7          ' Compare against commands
8          MOV     ptrb, #command_string
9          MOV     ptrb, ##cmd_help
10         CALL    #string_compare
11  if_z JMP     #help_command
12
13         MOV     ptrb, ##cmd_run
14         CALL    #string_compare
15  if_z JMP     #run_command
16
17         ' Many more commands...
18
19  help_command
20         ' 500 instructions of help text display
21         RET
22
23  run_command
24         ' 1000 instructions of program execution
25         RET
26
27  string_compare
28         ' 50 instructions of string comparison
29         RET
```



```
30
31 ' Thousands of instructions total
32 ' Would need multiple COGs without hub execution!
```

When to Use Hub Execution

Perfect for:

- User interfaces and menus
- Command processors
- Complex algorithms
- String manipulation
- Protocol handlers
- Error handling and recovery

Avoid for:

- Interrupt handlers (if you use them)
- Precise timing loops
- Bit-banged protocols
- Real-time control loops

What We've Learned

You've mastered hub execution:

- Understanding COG vs hub trade-offs
- Automatic mode switching
- Mixing COG and hub code
- FIFO streaming of instructions
- When to use each mode
- Real-world applications

Coming Up Next

Chapter 11 tackles the controversial topic: “Why No Interrupts?” We’ll explore why the Propeller philosophy says you don’t need them, and why that’s actually a good thing!

Have Fun! Hub execution is like having a sports car that can also carry furniture - you get both speed and capacity when you need them!

Chapter 11: Why No Interrupts?

The most controversial P2 feature explained

The Hook: Interrupts Without Interrupts

Here's a traditional interrupt-driven button handler:

```
1 // Traditional approach (NOT P2!)
2 ISR(BUTTON_INTERRUPT) {
3     // Interrupt service routine
4     buttonPressed = true;
5     // Return to interrupted code
6 }
```

And here's the P2 way:

```
1 ' Dedicated COG watching button
2 button_watcher
3     TESTP    #BUTTON_PIN wc
4     if_c WRLONG  ##1, ##button_flag
5     JMP      #button_watcher
6
7 ' Main COG doing important work
8 main_code
9     ' Never interrupted!
10    ' Checks button_flag when convenient
```

No interrupt latency. No context switching. No priority inversion. No critical sections. Just clean, deterministic, parallel processing.

The Interrupt Problem

Let me tell you a story. You're concentrating on a complex problem when someone taps your shoulder. You stop, handle their request, then try to remember where you were. Now imagine this happening randomly, unpredictably, dozens of times per second.

That's interrupts.

Traditional processors need interrupts because they only have one processor. Something important happens? Stop everything and handle it! But this causes:

- **Latency:** Time to save context and jump to handler
- **Jitter:** Variable response time depending on what was interrupted
- **Priority inversion:** Low-priority task blocks high-priority
- **Race conditions:** Shared data access problems
- **Debugging nightmares:** Non-reproducible timing bugs

The Propeller Solution

Eight processors. No sharing required.

```
1  ' COG 0: Main application
2  main_app
3      ' Complex calculations
4      ' Never interrupted
5      RDLONG  command, ##mailbox wz
6      if_nz CALL  #process_command
7      JMP     #main_app
8  ' COG 1: Serial port handler
9  serial_handler
10     ' Continuously monitors serial
11     TESTP   #RX_PIN wc
12     if_c CALL  #receive_byte
13     JMP     #serial_handler
14
15  ' COG 2: Motor control
16  motor_control
17     ' Precise timing loops
18     ' Never disrupted
19     WAITCNT motor_time
```

```

20          DRVNOT  #STEP_PIN
21          JMP     #motor_control
22
23  ' COG 3: Sensor monitor
24  sensor_monitor
25      ' Watches multiple sensors
26      ' Responds instantly
27      ' ... and so on

```

Each COG does one thing perfectly. No interruptions. No conflicts. Just pure, focused execution.

Real-World Example: Perfect Servo Control

With interrupts, servo pulses jitter. With dedicated COGs, they're perfect:

```

1  ' COG dedicated to servo control
2  servo_cog
3      GETCT    pulse_time
4
5  servo_loop
6      ' Generate 8 servo pulses simultaneously
7      MOV     servo_mask, ##$FF      ' 8 servos
8      OR      outa, servo_mask      ' All high
9
10     MOV     index, #0
11  check_servos
12     RDLONG   width, ptra[index]     ' Get pulse width
13     ADDCT1   pulse_time, width      ' Set compare time
14
15     WAITCT1                                     ' Wait for exact time
16     BITL     outa, index             ' Turn off this servo
17
18     INCMOD   index, #7
19     TJNZ     servo_mask, #check_servos
20
21     ' Wait for 20ms frame
22     WAITX    ##4_000_000
23     JMP     #servo_loop
24

```

```
25 ' Result: 8 servos with ZERO jitter!
```

Try that with interrupts. I'll wait. Actually, I won't - it's impossible to achieve this precision with interrupts.

“But P2 HAS Interrupts!”

Yes, it does. And you probably shouldn't use them.

Well, let me be more nuanced. P2 has interrupts for those rare cases where you absolutely need them:

```
1 ' Setting up an interrupt (not recommended!)
2     SETSE1  %#001<<6 + PANIC_BUTTON    ' SE1 triggers when pin goes high
3     SETINT1 #EVENT_SE1                  ' Enable INT1 on SE1 event
4 int1_handler
5     ' Interrupt code here
6     RETI1
```

When might you use them?

- Porting legacy code that requires interrupts
- Ultra-low-power designs where COGs must sleep
- Theoretical minimum latency response (but dedicated COG is usually faster!)

Uff! Even writing interrupt code feels wrong on a Propeller!

The Medicine Cabinet

Still thinking you need interrupts? Here's your medicine:

Think you need an interrupt for...

Fast response?

```

1 ' Dedicated COG responds in ~4 clocks
2 watcher
3     TESTP    #INPUT_PIN wz          ' Test pin state
4     if_nz JMP    #watcher           ' Loop until pin high
5     DRVH     #RESPONSE_PIN          ' Instant response!
```

Multiple events?

```

1 ' One COG watches everything
2 monitor
3     TEST     sensors, #SENSOR1 wz
4     if_nz CALL #handle_sensor1
5     TEST     sensors, #SENSOR2 wz
6     if_nz CALL #handle_sensor2
7     ' Check all sensors every loop
```

Periodic tasks?

```

1 ' Perfect timing without interrupts
2     GETCT    next_time
3 .loop ADDCT1  next_time, ##PERIOD
4     WAITCT1                      ' Exact timing
5     CALL     #periodic_task
6     JMP      #.loop
```

See? No interrupts needed!

The Event System: Better Than Interrupts

P2 has something better than interrupts - events:

```

1  ' Configure event to watch pin
2      SETSE1  #%01_000000 | BUTTON_PIN  ' Rising edge event
3
4  ' Main code runs normally
5  main_loop
6      ' Do work...
7      POLLSE1 wc          ' Check if event occurred
8      if_c CALL    #handle_button  ' Handle when convenient
9      ' Continue work...
10     JMP      #main_loop

```

Events are like interrupts that wait politely for you to check them. No rudeness!

Interrupt Horror Stories

Let me share why we avoid interrupts:

Story 1: The Jittery Display

Approach	Problem	Result
With Interrupts	Display updates interrupted by serial	Visible glitches, tearing, inconsistent timing
With COGs	Display COG runs uninterrupted	Perfect, smooth, glitch-free display

Story 2: The Missed Pulse

Approach	Problem	Result
With Interrupts	Motor step interrupted by sensor read	Missed step, motor stalls, position lost
With COGs	Motor COG never misses a beat	Perfect positioning, no lost steps

Story 3: The Debugging Nightmare

Approach	Problem	Result
With Interrupts	Bug only appears under specific timing	Days of debugging, hair loss, coffee overdose
With COGs	Deterministic timing, reproducible behavior	Bug found in minutes, sanity preserved

Your Turn: COG vs Interrupt Challenge

Your Turn: Build a reaction timer without interrupts

Starting code:

```

1      ORG      0
2
3  ' COG 0: Main game logic
4      SETQ     ##button_flag          ' PTRA for new COG
5      COGINIT #1, @button_watcher
6
7  game_loop
8      ' Random delay
9      GETRND   delay
10     WAITX    delay
11
12     DRVH     #LED_PIN
13     WRLONG   #0, ##button_flag
14     GETCT    start_time
15
16  ' Wait for button (no interrupt!)
17  wait_press
18     RDLONG   pressed, ##button_flag wz
19     if_z JMP     #wait_press
20
21     GETCT    end_time
22     SUB      end_time, start_time
23     ' Display reaction time
24
25  ' COG 1: Button watcher
26     ORGH     $400

```



```
27 button_watcher
28     ' Your code here
```

Goal: Implement button watcher COG Hint: Continuously monitor and set flag Success Check: Perfect timing without interrupts

The Philosophy Deep Dive

The Propeller philosophy is about **determinism over responsiveness**.

Traditional processors optimize for average-case performance:

- Interrupts handle rare events
- Most code runs uninterrupted
- When events happen, everything stops

Propeller optimizes for worst-case determinism:

- Every COG runs predictably
- No surprises, ever
- Timing is guaranteed

It's the difference between a talented soloist who might miss a note and an orchestra where everyone plays their part perfectly.

When Interrupts Actually Make Sense

I'll admit it - there are rare cases where interrupts are appropriate:

1. **Power-critical applications** where COGs must sleep
2. **Legacy code ports** that fundamentally require interrupts
3. **Single-COG designs** (but why waste the P2's power?)

But in 15 years of Propeller programming, I've needed interrupts exactly... never.

Common “But What About...” Questions

Q: “But what about interrupt priority?” A: COGs don’t have priority. They’re all equal. Design your system accordingly.

Q: “How do I handle critical events?” A: Dedicate a COG to critical events. It will respond faster than any interrupt.

Q: “Isn’t dedicating a whole COG wasteful?” A: You have eight! And a focused COG is simpler than interrupt-riddled code.

Q: “What about power consumption?” A: Use WAITSE/WAITCT for low-power waiting. COG sleeps until event.

What We’ve Learned

You now understand the Propeller way:

- Why interrupts cause problems
- How COGs eliminate interrupt need
- Event system as polite alternative
- Real-world benefits of no interrupts
- Rare cases where interrupts might be used
- The philosophy of determinism

Coming Up Next

Chapter 12 shows you “Optimization Mastery” - how to make your PASM2 code blazingly fast. We’ll explore the pipeline, instruction pairing, and timing tricks that squeeze every drop of performance from the P2.

Have Fun! And remember - in a world of interruptions, be a COG: focused, deterministic, and uninterruptible!

Chapter 12: Optimization Mastery

Making the fast faster

The Hook: Double Your Speed with One Change

Look at this seemingly innocent code:

```
1  ' Before optimization: 13 clocks
2  .loop  RDLONG  value, ptra      ' 9-16 clocks hub access
3          ADD    value, #1        ' 2 clocks
4          WRLONG  value, ptra      ' 3-10 clocks
5          ADD    ptra, #4          ' 2 clocks
6          DJNZ   count, #.loop    ' 2/4 clocks
7  ' After optimization using PTR expressions:
8  .loop  RDLONG  value, ptra      ' Read from current address
9          ADD    value, #1        ' Process
10         WRLONG  value, ptra++    ' Write and increment in one!
11         DJNZ   count, #.loop    ' Saved the ADD instruction
```

Almost twice as fast! The secret? Understanding how P2 really works.

Understanding the Pipeline

P2 has a 2-stage pipeline:

1. **Fetch** - Get next instruction
2. **Execute** - Do the work

This means while one instruction executes, the next is already being fetched:

```
1          ADD    x, y            ' Executing while next inst fetches
2          SUB    a, b            ' Fetching while previous executes
3          ' Perfect overlap = maximum throughput
```

Instruction Timing Basics

Not all instructions are created equal:

```

1  ' 2-clock instructions (most ALU operations)
2      ADD      x, y          ' 2 clocks
3      MOV      a, b          ' 2 clocks
4      AND      c, d          ' 2 clocks
5  ' Variable timing (hub access)
6      RDLONG   value, hubaddr ' 9-16 clocks (hub slot wait)
7      WRLONG   value, hubaddr ' 3-10 clocks (variable)
8
9  ' Long operations (CORDIC)
10     QROTATE  x, angle      ' 2 clocks to start
11     GETQX    result        ' 2 clocks (but wait 55 for result)
12
13  ' Special cases
14     MUL      x, y          ' 2 clocks
15     QDIV     x, y          ' 2 clocks to start
16     GETQX    result        ' 2 clocks (but wait 30 for result)

```

REP: The Speed Loop

REP creates hardware-accelerated loops with zero overhead:

```

1  ' Traditional loop: overhead per iteration
2  .loop  ADD      sum, value    ' 2 clocks
3         ADD      ptr, #4       ' 2 clocks
4         DJNZ     count, #.loop ' 2 or 4 clocks (4 if branch taken)
5  ' REP loop: 0 clocks overhead!
6         REP      #2, count     ' Repeat next 2 instructions
7         ADD      sum, value    ' 2 clocks
8         ADD      ptr, #4       ' 2 clocks = 4 total!

```

That's 33% faster just by using REP!

SKIP: Conditional Execution on Steroids

SKIP and SKIPF let you conditionally execute patterns of instructions:

```

1 ' Traditional: multiple jumps
2     CMP    x, #5 wcz
3 if_a     JMP    #greater
4 if_b     JMP    #less
5         JMP    #equal
6 ' With SKIP: no jumps!
7     CMP    x, #5 wcz
8     SKIP   ##%11000      ' Skip pattern based on flags
9     MOV    result, #1     ' Execute if equal
10    MOV    result, #2     ' Execute if less
11    MOV    result, #3     ' Execute if greater
12    ' No pipeline stalls from jumps!

```

Hub Access Optimization

Hub timing is critical for performance:

```

1 ' Unaligned hub access: variable timing
2     RDLONG v1, ##$1001    ' Not long-aligned, slower
3
4 ' Aligned hub access: predictable timing
5     RDLONG v1, ##$1000    ' Long-aligned, faster
6
7 ' Sequential access: maximum speed
8     RDLONG v1, ptr++      ' Hardware manages sequence
9     RDLONG v2, ptr++      ' Optimal hub slot usage
10    RDLONG v3, ptr++      ' Maximum throughput

```

The FIFO Fast Path

For ultimate speed, use the FIFO:

```

1 ' Traditional hub reading: ~6 clocks average per long
2 .loop RDLONG value, ptr++

```

```

3      ADD      sum, value
4      DJNZ     count, #.loop
5  ' FIFO reading: 2 clocks per long!
6      RDFAST   #0, ptra      ' Start FIFO
7  .loop RFLONG  value        ' 2 clocks, always!
8      ADD      sum, value    ' 2 clocks
9      DJNZ     count, #.loop ' 2 clocks
10     ' 3x faster for sequential reads!

```

Parallel Operations

CORDIC operations can overlap with other work:

```

1  ' CORDIC overlaps with other instructions
2      QMUL     x, y          ' Start 32x32->64 multiply (CORDIC)
3      ' 55 clocks to do other work!
4      ADD      a, b          ' These execute during CORDIC
5      SUB      c, d
6      MOV      index, #0
7      RDLONG   data, ptra++
8      ' ... more work
9      GETQX    low_result    ' Get CORDIC result (lower 32 bits)
10     GETQY    high_result   ' Get CORDIC result (upper 32 bits)
11  ' QROTATE overlap
12     QROTATE  x_coord, angle ' Start rotation (D=X, S=angle)
13     ' 55 clocks of other work!
14     GETQX    new_x         ' Get rotated X
15     GETQY    new_y         ' Get rotated Y

```

Note: MUL/MULS are 2-clock ALU instructions that complete immediately (16x16->32). Use QMUL for 32x32->64 with CORDIC overlap.

Real-World Example: Fast Memory Copy

Let's optimize a memory copy routine:

```
1  ' Version 1: Basic (slow)
2  copy_basic
3      RDLONG  temp, source
4      WRLONG  temp, dest
5      ADD     source, #4
6      ADD     dest, #4
7      DJNZ    count, #copy_basic
8      ' ~13 clocks per long
9  ' Version 2: Better pointers
10 copy_better
11      RDLONG  temp, ptrb++
12      WRLONG  temp, ptrb++
13      DJNZ    count, #copy_better
14      ' ~8 clocks per long
15
16 ' Version 3: Block transfer (ultimate)
17 copy_ultimate
18      SUB     count, #1      ' SETQ needs count-1 (0 = 1 long)
19      SETQ    count          ' Setup block transfer
20      RDLONG  buffer, source ' Read all at once
21      SETQ    count          ' (count already decremented)
22      WRLONG  buffer, dest   ' Write all at once
23      ' <1 clock per long for large blocks!
```

The Medicine Cabinet

Optimization overwhelming you? Start with these simple improvements:

Three easy wins:

1. Use **PTRA/PTRB** instead of manual pointer math

```

1  ' Slow
2      RDLONG  x, addr
3      ADD     addr, #4
4  ' Fast
5      RDLONG  x, ptra++

```

2. Align your data to long boundaries

```

1      ALIGNL          ' Force long alignment
2 data  LONG          $12345678

```

3. Use **REP** for tight loops

```

1      REP      #1, count
2      ADD      sum, ptra++

```

Just these three changes often double performance!

Your Turn: Optimization Challenges

Your Turn: Optimize a checksum calculator

Starting code:

```

1  ' Slow version
2  checksum_slow
3      MOV      sum, #0
4      MOV      addr, ##buffer
5      MOV      count, #256
6  .loop  RDBYTE  temp, addr
7      ADD      sum, temp
8      ADD      addr, #1
9      DJNZ     count, #.loop

```


Goal: Make it at least 4x faster Hint: Read longs instead of bytes, use FIFO Success Check: Same checksum, much faster

Advanced Techniques

Instruction Pairing

Some instruction pairs execute specially:

```

1 ' ## syntax handles AUGS automatically
2     MOV    x, ##$12345678 ' Assembler generates AUGS + MOV
3     ' Same result, cleaner code!
4
5 ' ALTD + instruction = indirect addressing
6     ALTD    index, #array
7     MOV     0-0, value    ' Stores to array[index]
```

Pipeline-Aware Coding

Avoid pipeline stalls:

```

1 ' Bad: result needed immediately
2     ADD     x, y
3     CMP     x, #10 wcz    ' Stall waiting for x
4
5 ' Good: interleave operations
6     ADD     x, y
7     MOV     a, b          ' Do something else
8     CMP     x, #10 wcz    ' Now x is ready
```

Unrolling Loops

Sometimes removing the loop is faster:

```

1 ' Looped version
2     REP     #1, #4
3     ADD     sum, ptr++
4
5 ' Unrolled version (faster for small counts)
```

```
6      ADD    sum, ptr++
7      ADD    sum, ptr++
8      ADD    sum, ptr++
9      ADD    sum, ptr++
```

Common Optimization Gotchas

1. **Premature optimization** - Get it working first, then optimize
2. **Over-optimizing** - Sometimes clarity is worth 2 clocks
3. **Ignoring the big picture** - Optimize the bottleneck, not everything
4. **Breaking functionality** - Fast but wrong is useless
5. **Forgetting about power** - Faster isn't always better for battery life

Profiling and Measurement

Always measure your optimizations:

```
1  ' Time your code
2      GETCT    start_time
3
4      ' Code to measure
5      CALL     #function_to_test
6
7      GETCT    end_time
8      SUB      end_time, start_time
9      ' end_time now contains exact clock cycles
```

What We've Learned

You're now an optimization expert:

- Understanding the P2 pipeline
- Instruction timing knowledge
- REP and SKIP for zero-overhead loops
- FIFO for maximum throughput
- Parallel operation techniques
- Real-world optimization strategies

Coming Up Next

Chapters 13-15 provide quick examples of Video Generation, Serial Protocols, and Signal Processing - with references to dedicated manuals for deep dives. Think of them as appetizers showing what's possible!

Have Fun! Remember, the best optimization is often a better algorithm. But when you need every last cycle, you now know how to get them!

Chapter 13: LUT Memory - Your Private Lookup Table

512 longs of fast, deterministic storage in every COG

The Hook: A Lookup Table in 3 Cycles

Need fast data lookup without hub timing? Every COG has its own private 512-long Lookup RAM (LUT):

```

1 ' Sine table lookup - 3 clocks, every time
2 get_sine
3     AND     angle, #$FF      ' Mask to table index
4     RDLUT   value, angle     ' Read from LUT in 3 clocks!
5     RET

```

No hub timing to worry about. No waiting for the egg beater. Just 3 clock cycles, guaranteed. The LUT is like having a personal data assistant that never takes a coffee break.

Why Another Memory?

You might be thinking, “Wait, I already have COG RAM and Hub RAM - why do I need a third memory?” Excellent question!

Memory	Size per COG	Access Time	Special Features
COG RAM	512 longs	2 clocks	Instructions live here
Hub RAM	512 KB shared	2-9 clocks (hub slot wait)	Shared by all COGs
LUT RAM	512 longs	3 clocks	Private, deterministic, shareable with neighbor

The LUT fills a sweet spot: faster than hub memory, doesn’t compete with your instruction space, and has a trick up its sleeve - neighboring COGs can share LUTs!

Reading and Writing the LUT

Basic LUT Access

```

1  ' Write to LUT
2      WRLUT    #$12345678, #100  ' Write constant to LUT[100]
3      WRLUT    value, index      ' Write variable to LUT[index]
4  ' Read from LUT
5      RDLUT    result, #100      ' Read LUT[100] into result
6      RDLUT    data, index       ' Read LUT[index] into data

```

Notice the operand order: **WRLUT** writes its first operand to the address in the second, while **RDLUT** reads from its second operand into the first. A bit backwards from what you might expect, but you'll get used to it.

Building a Lookup Table

Here's how to load a sine table into LUT:

```

1  ' Copy 256-entry sine table from hub to LUT
2  load_sine_table
3      MOV      index, #0
4      LOC      ptr, #\sine_data_hub  ' Hub address of table
5  .loop  RDLONG  value, ptr++        ' Read from hub
6      WRLUT    value, index         ' Write to LUT
7      ADD      index, #1
8      CMP      index, #256 wz
9  if_nz  JMP      #.loop
10     RET
11  ' Now lookups are fast!
12  get_sine
13     RDLUT     sine_value, angle    ' 3 clocks!
14     RET

```

Sidetrack

Bulk LUT Loading with SETQ2

For loading entire tables, **SETQ2** + **RDLONG** can transfer hub data directly to LUT addresses \$200+:

```

1      SETQ2    #256-1          ' 256 longs
2      RDLONG   $200, hub_table_ptr ' Load into LUT from $200

```

This works because the assembler maps LUT addresses \$200-\$3FF. Just remember the -1 in **SETQ2** (same rule as **SETQ** for hub block transfers).

LUT Sharing Between COGs

Here's something clever: adjacent COG pairs can share LUT data! When you enable LUT sharing with **SETLUTS**, writes your neighbor makes to their LUT are automatically *copied* to your LUT too.

```

1 ' --- COG 1 (consumer) - MUST enable sharing FIRST ---
2     SETLUTS #1          ' Enable LUT write copying FROM COG 0
3     ' Now when COG 0 writes to its LUT, data is COPIED to our LUT
4 ' --- COG 0 (producer) - writes AFTER consumer enables sharing ---
5     WRLUT    message, #10 ' Write MY LUT[10] (copies to COG 1)
6     WRLUT    #1, #0       ' Set ready flag (copies to COG 1)
7 ' --- COG 1 (consumer) - reads its OWN LUT (which contains copies) ---
8 .wait  RDLUT   flag, #0    ' Read MY LUT[0] (contains copy from COG 0)
9     CMP      flag, #1 wz
10    if_nz    JMP      #.wait
11    RDLUT    message, #10  ' Read MY LUT[10] (copied from COG 0)

```

The key instruction is:

- **SETLUTS**: Enable write copying - when neighbor writes with **WRLUT**, data is copied to YOUR LUT
- **RDLUT**: Read your own LUT (which now contains copied data)

Important: The consumer COG must enable **SETLUTS** *before* the producer writes, otherwise the writes won't be copied!

This gives you a 512-long shared buffer between COG pairs without touching hub memory. Perfect for high-bandwidth data passing!

Sidetrack**Which COGs Are Neighbors?**

The LUT sharing pairs are fixed:

- COG 0 COG 1
- COG 2 COG 3
- COG 4 COG 5
- COG 6 COG 7

An even-numbered COG reads its odd neighbor's LUT, and vice versa. You cannot read LUTs from non-adjacent COGs.

Practical Examples**Fast Data Transformation**

```

1 ' Gamma correction table in LUT
2 ' Input: 8-bit value in 'pixel'
3 ' Output: Gamma-corrected value
4 gamma_correct
5     AND    pixel, #$FF    ' Mask to 8 bits
6     RDLUT  pixel, pixel    ' Transform via table
7     RET
8 ' Initialize gamma table (power law curve)
9 ' Would be pre-calculated and loaded from hub

```

Circular Buffer in LUT

```

1 ' Fast circular buffer using LUT
2 ' 256-entry buffer at LUT addresses 0-255
3 buf_write_ptr    LONG    0
4 buf_read_ptr     LONG    0
5 put_byte
6     WRLUT    data, buf_write_ptr
7     ADD      buf_write_ptr, #1
8     AND      buf_write_ptr, #$FF    ' Wrap at 256
9     RET
10 get_byte

```

```

11      RDLUT    data, buf_read_ptr
12      ADD      buf_read_ptr, #1
13      AND      buf_read_ptr, #$FF      ' Wrap at 256
14      RET

```

Fast Stack in LUT

```

1  ' Stack implementation in LUT
2  ' Grows downward from $1FF
3  stack_ptr      LONG    $1FF
4  PUSH
5      WRLUT    value, stack_ptr
6      SUB      stack_ptr, #1
7      RET
8  POP
9      ADD      stack_ptr, #1
10     RDLUT    value, stack_ptr
11     RET

```

LUT with the Streamer

Here's where LUT gets really interesting. The Streamer can read directly from LUT to generate waveforms without any COG intervention:

```

1  ' Fill LUT with waveform data
2  ' Then let Streamer output it to DAC
3  load_waveform
4      MOV      index, #0
5  .fill  MOV      value, index
6      SHL      value, #24      ' Scale for DAC
7      WRLUT    value, index
8      ADD      index, #1
9      CMP      index, #512 wz
10     if_nz JMP      #.fill
11     ' Now configure Streamer to read from LUT
12     ' Streamer handles the rest - no COG cycles needed!

```

The Streamer configuration for LUT reading is covered in detail in the Video and Audio manuals -

but the key point is that your LUT becomes a 512-sample waveform buffer that plays automatically.

Common Gotchas

```

1 ' WRONG - This reads COG RAM, not LUT!
2     MOV     value, $200      ' $200 is COG RAM address

```

RIGHT: Use RDLUT for LUT access

```

1 ' RIGHT - RDLUT addresses the LUT space
2     RDLUT   value, #0        ' LUT address 0

```

```

1 ' WRONG - No data to read yet!
2     SETLUTS #1                ' Enable sharing
3     RDLUT   data, #10         ' Empty - neighbor hasn't written!

```

RIGHT: Wait for neighbor's write signal

```

1 ' RIGHT - Wait for data to be copied
2     SETLUTS #1                ' Enable sharing BEFORE neighbor writes
3 .wait RDLUT   ready, #0        ' Check flag in MY LUT
4     TJZ     ready, #.wait      ' Wait until neighbor writes
5     RDLUT   data, #10         ' Now MY LUT has copied data

```

Medicine Cabinet

The Medicine Cabinet

LUT Memory Quick Reference

Instruction	Operation	Cycles
RDLUT D, S	Read LUTS into D	3
WRLUT D, S	Write D to LUTS	2
SETLUTS D	Enable LUT write copying (D[0]=1)	2

Memory Map:

- LUT addresses: 0-511 (512 longs = 2KB)
- Neighbor pairs: 0 1, 2 3, 4 5, 6 7

Best Uses:

- Lookup tables (sine, gamma, encoding)
- Fast circular buffers
- COG-pair data sharing
- Streamer waveform source

Your Turn

Your Turn

Exercise 1: Build an 8-bit Encoder

Create a LUT-based ASCII to 7-segment display encoder. Load a 128-entry table where each entry maps an ASCII code to the 7-segment pattern for that character.

```

1 ' Your code here:
2 ' 1. Load segment patterns into LUT
3 ' 2. Write encode_char routine
4 ' Hint: rdlut segment_pattern, ascii_char

```

Your Turn

Exercise 2: High-Speed COG Communication

Use LUT sharing to create a message passing system between COG 2 and COG 3:

- COG 2 writes 8-long messages

- COG 3 reads them without hub access
- Use a simple ready/ack protocol

```
1 ' Hint: Use LUT[0] as ready flag, LUT[1-8] as message buffer
```

Continue to Chapter 14: Smart Pins Orientation →

Chapter 14: Smart Pins Orientation

64 autonomous I/O processors waiting to do your bidding

The Hook: A UART in 4 Lines

Remember that tedious bit-bang serial from Chapter 8? Watch this:

```
1  ' Configure pin as UART transmitter - done!
2      DIRL      #TX_PIN          ' Reset pin first!
3      WRPIN     ##P_ASYNC_TX, #TX_PIN ' Configure as async TX
4      WXPIN     ##BAUD_115200, #TX_PIN ' Set baud rate
5      DIRH      #TX_PIN          ' Enable - runs on its own
```

That's it. The pin is now a fully autonomous UART transmitter. It handles start bits, stop bits, timing - everything. You just feed it bytes with **WYPIN** and it sends them. The pin has become a state machine.

And here's the mind-bending part: *every single one of the 64 pins can do this*. Or PWM. Or ADC. Or quadrature decoding. Or 28 other modes.

What Are Smart Pins, Really?

Each of the P2's 64 I/O pins contains its own little processor - a state machine that can operate completely independently of the COGs. This means:

- A pin configured as UART keeps sending/receiving without COG intervention
- A PWM output keeps running its duty cycle automatically
- An ADC samples continuously in the background
- A quadrature decoder tracks position even while your COG does other things

The COG only needs to configure the pin and occasionally read/write data. The pin does the rest.

The Universal Smart Pin Pattern

Every Smart Pin follows the same configuration pattern. This is **the most important thing to remember**:

```

1  ' === THE SMART PIN RECIPE ===
2  ' Step 1: RESET the pin (CRITICAL!)
3      DIRL    pin          ' Always start by resetting
4  ' Step 2: CONFIGURE the mode
5      WRPIN   mode, pin    ' What should this pin do?
6  ' Step 3: SET parameters
7      WXPIN   x_value, pin  ' Mode-specific parameter X
8      WYPIN   y_value, pin  ' Mode-specific parameter Y
9  ' Step 4: ENABLE the pin
10     DIRH    pin          ' Start the magic!

```

Sidetrack

Why DIRL First?

The **DIRL** at the start isn't optional politeness - it's *required*. Smart Pins must be reset before configuration to ensure they're in a known state. Skip this and you'll get unpredictable behavior as old settings conflict with new ones.

Think of it like power-cycling a misbehaving device. Always start fresh.

The Core Instructions

Configuration Instructions

Instruction	Purpose
WRPIN mode, pin	Set the operating mode
WXPIN value, pin	Set X parameter (mode-specific)
WYPIN value, pin	Set Y parameter (mode-specific)
DIRH pin	Enable the Smart Pin
DIRL pin	Disable/reset the Smart Pin

Data Instructions

Instruction	Purpose
WYPIN data, pin	Write data to Smart Pin (same instruction!)
RDPIN data, pin	Read result, clear "ready" flag

Instruction	Purpose
RQPIN data, pin	Read result, keep “ready” flag
AKPIN pin	Acknowledge (clear “ready” flag only)

Status Instructions

Instruction	Purpose
TESTP pin WC	Check if IN flag is set (data ready)
TESTPN pin WC	Check if IN flag is clear

Understanding the IN Flag

Every Smart Pin has an IN flag that signals “something happened.” What that something is depends on the mode:

- **UART TX:** IN high = ready for another byte
- **UART RX:** IN high = byte received
- **ADC:** IN high = new sample ready
- **PWM:** IN high = period complete
- **Counter:** IN high = threshold reached

You check this flag with **TESTP** and clear it by reading with **RDPIN** (or explicitly with **AKPIN**).

```

1 ' Wait for Smart Pin to be ready
2 wait_ready
3     TESTP    #PIN wc        ' Check IN flag
4     if_nc JMP    #wait_ready ' Loop if not ready
5     RDPIN    data, #PIN     ' Read and clear flag

```

Sidetrack

Event-Driven Alternative

Instead of polling with **TESTP**, you can use the event system:

```

1 SETSE1  #001<<6 + PIN    ' Event when IN rises
2 WAITSE1                                ' Sleep until ready - no polling!
3 RDPIN   result, #PIN      ' Read the result

```

This is more efficient because your COG sleeps instead of spinning. See Chapter 15 for the full event story.

Common Smart Pin Modes

Here are the modes you'll use most often:

Asynchronous Serial (UART)

```

1 ' Transmit mode
2     DIRL    #TX_PIN
3     WRPIN   ##P_ASYNC_TX | P_OE, #TX_PIN
4     WXPIN   ##(clkfreq/ baud)<<16 | 7, #TX_PIN ' Baud + 8 bits
5     DIRH    #TX_PIN
6 ' Send a byte
7 send    TESTP   #TX_PIN wc      ' Wait for ready
8   if_nc JMP     #send
9     WYPIN   BYTE, #TX_PIN      ' Send it

```

```

1 ' Receive mode
2     DIRL    #RX_PIN
3     WRPIN   ##P_ASYNC_RX, #RX_PIN
4     WXPIN   ##(clkfreq/ baud)<<16 | 7, #RX_PIN
5     DIRH    #RX_PIN
6 ' Get a byte
7 recv    TESTP   #RX_PIN wc      ' Check for received byte
8   if_nc JMP     #recv
9     RDPIN   BYTE, #RX_PIN      ' Get it
10    SHR     BYTE, #24          ' Shift to low byte

```

PWM Output

```

1 ' PWM mode - period + duty cycle
2     DIRL    #PWM_PIN
3     WRPIN   ##P_PWM_SAWTOOTH | P_OE, #PWM_PIN
4     WXPIN   ##period, #PWM_PIN      ' Period in clocks
5     WYPIN   ##duty, #PWM_PIN        ' High time in clocks
6     DIRH    #PWM_PIN
7 ' Change duty cycle on the fly
8     WYPIN   ##new_duty, #PWM_PIN    ' Just update Y parameter

```

ADC Input

```

1 ' ADC mode - continuous sampling
2     DIRL    #ADC_PIN
3     WRPIN   ##P_ADC | P_ADC_GIO, #ADC_PIN
4     WXPIN   ##14, #ADC_PIN          ' 14-bit mode
5     DIRH    #ADC_PIN
6 ' Read ADC value
7 read_adc
8     RDPIN   adc_value, #ADC_PIN      ' Get sample
9     SHR     adc_value, #17           ' Right-justify the result

```

Quadrature Encoder

```

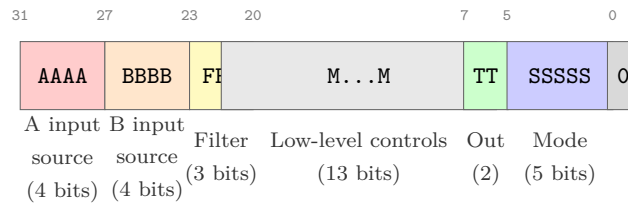
1 ' Quadrature decoder - A on pin, B on pin+1
2     DIRL    #ENC_PIN
3     WRPIN   ##P_QUADRATURE, #ENC_PIN
4     DIRH    #ENC_PIN
5 ' Read position
6     RDPIN   position, #ENC_PIN       ' Get accumulated count

```

Configuration Values Demystified

The mode values like `P_ASYNC_TX` are constants defined by the assembler. Here's what's happening behind the scenes:

The **WRPIN** D value is a 32-bit configuration:

WRPIN Configuration (32 bits):

For most common modes, you'll use predefined constants like `P_ASYNC_TX`, `P_PWM_SAWTOOTH`, `P_ADC`. The P2 assembler knows all of them.

Common Gotchas**WRONG: Forgetting to reset before configure**

```

1 ' WRONG - Pin may be in unknown state!
2     WRPIN    ##P_PWM_SAWTOOTH, #PIN
3     WXPIN    ##1000, #PIN
4     DIRH     #PIN

```

RIGHT: Always DIRL first

```

1 ' RIGHT - Start clean
2     DIRL     #PIN                    ' Reset first!
3     WRPIN    ##P_PWM_SAWTOOTH, #PIN
4     WXPIN    ##1000, #PIN
5     DIRH     #PIN

```

WRONG: Enabling before configuring

```

1 ' WRONG - Pin enabled with partial config!
2     DIRL     #PIN
3     DIRH     #PIN                    ' Enabled too early!
4     WRPIN    ##P_ASYNC_TX, #PIN

```

RIGHT: DIRH comes last

```

1 ' RIGHT - Configure completely, then enable
2     DIRL     #PIN
3     WRPIN    ##P_ASYNC_TX, #PIN

```

```

4      WXPIN    ##BAUD, #PIN
5      DIRH     #PIN                ' Enable last!

```

Medicine Cabinet

The Medicine Cabinet

Smart Pin Quick Reference

The Recipe:

1. **DIRL** pin — Reset the pin first
2. **WRPIN** mode, pin — Set the operating mode
3. **WXPIN** x, pin — Set X parameter
4. **WYPIN** y, pin — Set Y parameter
5. **DIRH** pin — Enable the Smart Pin

Common Modes:

- **UART TX**: P_ASYNC_TX — Serial transmit
- **UART RX**: P_ASYNC_RX — Serial receive
- **PWM**: P_PWM_SAWTOOTH — Sawtooth wave output
- **PWM**: P_PWM_TRIANGLE — Triangle wave output
- **ADC**: P_ADC — Analog input
- **Quadrature**: P_QUADRATURE — Encoder
- **NCO**: P_NCO_FREQ — Frequency output

Data Flow:

- **WYPIN** = Write data TO Smart Pin
- **RDPIN** = Read data FROM Smart Pin (clears IN)
- **TESTP** = Check if IN flag set

Golden Rule: DIRL before WRPIN, DIRH after WXPIN/WYPIN

Your Turn

Your Turn

Exercise 1: PWM LED Dimmer

Create a PWM output that dims an LED:

1. Configure a pin for PWM sawtooth mode
2. Set a 1 kHz period (at 160 MHz: period = 160,000)

3. Vary duty cycle from 0% to 100%

```
1 ' Your code here:  
2 ' Hint: Change duty with WYPIN new_duty, #LED_PIN
```

Your Turn

Exercise 2: Simple Serial Echo

Set up UART at 115200 baud:

1. Configure RX on pin 63
2. Configure TX on pin 62
3. Echo every received byte back

```
1 ' Your code here:  
2 ' At 160 MHz: baud_divisor = 160_000_000 / 115200 = 1389  
3 ' WXPIN format: (divisor << 16) | (bits - 1)
```

Going Deeper: This chapter covered the Smart Pin essentials - the configuration pattern and common modes. For complete coverage of all 32 modes, timing diagrams, and advanced techniques, see the dedicated “P2 Smart Pins Manual.”

Continue to Chapter 15: Event-Driven Programming →

Chapter 15: Event-Driven Programming

Stop spinning, start waiting

The Hook: No More Polling Loops

Remember all those busy loops waiting for things to happen?

```

1 ' OLD WAY: Spin waiting for serial data (burns CPU cycles!)
2 wait_rx TESTP    #RX_PIN wc      ' Check over and over
3   if_nc JMP      #wait_rx        ' Spin spin spin...
4       RDPIN     data, #RX_PIN
5 ' NEW WAY: Sleep until data arrives (zero CPU cycles!)
6       SETSE1    #%001<<6 + RX_PIN ' Wake on IN rise
7       WAITSE1
8       RDPIN     data, #RX_PIN

```

The event system lets your COG sleep while waiting. When the event happens, it wakes up instantly. No cycles wasted, and you respond the moment something happens.

Why Events Matter

Polling loops have two problems:

1. **They waste cycles** - The COG spins doing nothing useful
2. **They add latency** - You check periodically, so there's delay between "thing happened" and "you noticed"

The event system solves both. Your COG *sleeps* and *wakes the instant* something happens. It's like having a personal assistant tap your shoulder instead of constantly looking up to check.

The Four Selectable Events

Every COG has four configurable event channels: SE1, SE2, SE3, and SE4. Each can be configured to trigger on different conditions:

Event	Configuration	Wait	Poll
SE1	SETSE1	WAITSE1	POLLSE1

Event	Configuration	Wait	Poll
SE2	SETSE2	WAITSE2	POLLSE2
SE3	SETSE3	WAITSE3	POLLSE3
SE4	SETSE4	WAITSE4	POLLSE4

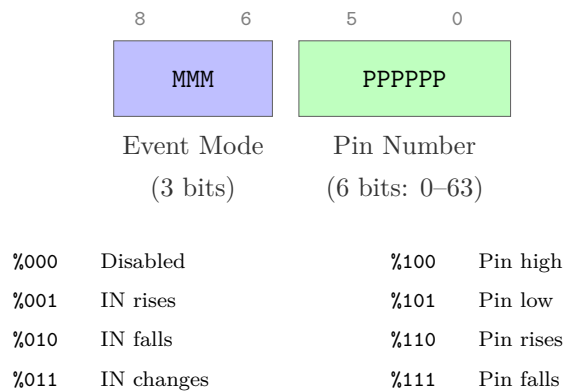
Plus there are built-in timer events:

Timer	Wait	Poll
CT1	WAITCT1	POLLCT1
CT2	WAITCT2	POLLCT2
CT3	WAITCT3	POLLCT3

Configuring an Event

The **SETSE1** through **SETSE4** instructions take a 9-bit configuration value:

SETSE1/2/3/4 Configuration (9 bits):



Event Modes

Mode	Meaning
%000	Never (disabled)
%001	IN rises (Smart Pin ready)
%010	IN falls
%011	IN changes

Mode	Meaning
%100	Pin high
%101	Pin low
%110	Pin rises
%111	Pin falls

EVENT_* Constants: When You Need Interrupts

While dedicated COGs are usually better than interrupts (see Chapter 11), sometimes you need them. The **SETINT1/2/3** instructions select which event triggers an interrupt using these constants:

Constant	Value	Description
EVENT_INT	%0000	Pin matches interrupt configuration
EVENT_CT1	%0001	CT equals CT1 (timer 1 target)
EVENT_CT2	%0010	CT equals CT2 (timer 2 target)
EVENT_CT3	%0011	CT equals CT3 (timer 3 target)
EVENT_SE1	%0100	Selectable event 1 triggered
EVENT_SE2	%0101	Selectable event 2 triggered
EVENT_SE3	%0110	Selectable event 3 triggered
EVENT_SE4	%0111	Selectable event 4 triggered
EVENT_PAT	%1000	SETPAT pattern detected
EVENT_FBW	%1001	Hub FIFO wrapped around
EVENT_XMT	%1010	Streamer needs data
EVENT_XFI	%1011	Streamer operation complete
EVENT_XR0	%1100	NCO frequency counter rolled
EVENT_XRL	%1101	Streamer matched pattern
EVENT_ATN	%1110	Another COG signaled attention
EVENT_QMT	%1111	CORDIC/PIX math complete

Using EVENT_* with SETINT:

```

1 ' Enable INT1 when SE1 event occurs
2     SETSE1  %#001<<6 + RX_PIN      ' SE1 = IN rise on RX_PIN
3     SETINT1 #EVENT_SE1              ' INT1 fires when SE1 triggers
4 ' Enable INT2 on timer match
5     ADDCT2  target, ##200_000      ' Set timer 2 target
6     SETINT2 #EVENT_CT2              ' INT2 fires when CT = CT2
7 ' Enable INT3 when another COG signals
8     SETINT3 #EVENT_ATN              ' INT3 fires on COGATN

```

Pro tip: You can also use these constants with WAITSE/POLLSE by first triggering them with the appropriate hardware condition, then waiting. But for most purposes, the SETSE mode bits (the table above this one) are what you'll configure directly.

Smart Pin Events

The most common use is waiting for a Smart Pin to have data:

```

1 ' Wait for Smart Pin on pin 15 to be ready
2     SETSE1  %#001<<6 + 15      ' IN rise on pin 15
3     WAITSE1                                ' Sleep until ready
4     RDPIN   data, #15           ' Get the data

```

Pin Edge Events

You can also wait for raw pin edges (without Smart Pin):

```

1 ' Wait for rising edge on pin 5
2     SETSE1  %#110<<6 + 5      ' Rise on pin 5
3     WAITSE1                                ' Sleep until edge
4     ' Edge detected!
5 ' Wait for falling edge on pin 10
6     SETSE2  %#111<<6 + 10     ' Fall on pin 10
7     WAITSE2
8     ' Edge detected!

```

Timer Events

For precise timing, use the counter comparison events:

```

1  ' Wait exactly 1 millisecond (at 200 MHz)
2      GETCT    target          ' Current time
3      ADD      target, ##200_000 ' +1ms at 200MHz
4      ADDCT1   target, #0       ' Set CT1 target
5      WAITCT1                      ' Sleep until CT >= CT1
6  ' Alternative using WAITX (simpler but less precise)
7      WAITX    ##200_000        ' Wait ~1ms at 200MHz

```

The timer events are:

- **ADDCT1/ADDCT2/ADDCT3:** Set the comparison target
- **WAITCT1/WAITCT2/WAITCT3:** Wait until CT reaches target
- **POLLCT1/POLLCT2/POLLCT3:** Check (non-blocking) if target reached

Waiting vs Polling

Two ways to use events:

WAIT - Sleep Until Event

```

1      WAITSE1                      ' COG sleeps here
2      ' Wakes instantly when event occurs

```

- COG sleeps, uses no cycles
- Wakes immediately when event fires
- Can't do anything else while waiting

POLL - Check and Continue

```

1      POLLSE1 wc                  ' Check event, clear if set
2  if_c JMP      #event_handler    ' Handle if occurred
3      ' Continue with other work...

```

- COG keeps running
- Checks event flag, clears it

- Returns result in C flag
- Good for servicing multiple events

Multiple Events

With four SE channels, you can monitor multiple sources:

```

1  ' Setup multiple events
2      SETSE1  %#001<<6 + RX_PIN      ' Serial data ready
3      SETSE2  %#110<<6 + BUTTON_PIN ' Button pressed
4      SETSE3  %#001<<6 + ADC_PIN     ' ADC sample ready
5  event_loop
6      POLLSE1 wc                      ' Check serial
7      if_c    CALL    #handle_serial
8      POLLSE2 wc                      ' Check button
9      if_c    CALL    #handle_button
10     POLLSE3 wc                      ' Check ADC
11     if_c    CALL    #handle_adc
12     JMP     #event_loop

```

Practical Examples

Timeout with Fallback

```

1  ' Wait for serial data, but give up after 100ms
2  wait_with_timeout
3      SETSE1  %#001<<6 + RX_PIN      ' Serial ready event
4      GETCT   timeout
5      ADD     timeout, ##16_000_000  ' 100ms at 160MHz
6      ADDCT1  timeout, #0
7  .wait  POLLSE1 wc                      ' Check serial
8      if_c    JMP     #.got_data
9      POLLCT1 wc                      ' Check timeout
10     if_c    JMP     #.timed_out
11     JMP     #.wait
12  .got_data
13      RDPIN   data, #RX_PIN
14      RET

```

```

15 .timed_out
16     MOV     data, #-1      ' Return error
17     RET

```

Debounced Button Press

```

1  ' Wait for clean button press with debounce
2  debounced_button
3      SETSE1  #%110<<6 + BUTTON  ' Rising edge
4      WAITSE1                                ' Wait for press
5      WAITX   ##2_000_000          ' 10ms debounce at 200MHz
6      TESTP   #BUTTON wc           ' Verify still pressed
7  if_nc JMP    #debounced_button  ' Bounce - try again
8      RET                                ' Clean press!

```

Precise Periodic Sampling

```

1  ' Sample ADC at exactly 10 kHz
2  sample_loop
3      GETCT   next_sample
4  .loop  ADDCT1 next_sample, ##16_000  ' 100us period
5      WAITCT1                                ' Wait for next slot
6      RDPIN   sample, #ADC_PIN          ' Read sample
7      WRLONG  sample, buffer_ptr        ' Store it
8      ADD     buffer_ptr, #4
9      JMP     #.loop

```

ATN - Inter-COG Events

The ATN (attention) system lets COGs signal each other:

```

1  ' COG 0: Signal another COG
2      COGATN  #%0000_0010          ' Send ATN to COG 1
3  ' COG 1: Wait for attention
4      WAITATN                                ' Sleep until ATN received
5      ' Another COG signaled us!

```

The **COGATN** instruction takes an 8-bit mask where each bit corresponds to a COG. Setting bit N sends attention to COG N.

Common Gotchas

```

1 ' WRONG - Event may fire before you're ready
2     SETSE1  #%001<<6 + PIN
3     ' ... do other stuff ...
4     WAITSE1                      ' May return immediately!
```

RIGHT: Poll first to clear any pending event

```

1 ' RIGHT - Clear any stale event
2     SETSE1  #%001<<6 + PIN
3     POLLSE1                      ' Clear if already set
4     ' ... do other stuff ...
5     WAITSE1                      ' Now wait cleanly
```

```

1 ' WRONG - Can only wait for one event at a time
2     WAITSE1                      ' Stuck here until SE1
3     ' SE2 might fire and be missed!
```

RIGHT: Use POLL loop for multiple events

```

1 ' RIGHT - Check all sources
2 .loop  POLLSE1 wc
3     if_c CALL    #handle_se1
4     POLLSE2 wc
5     if_c CALL    #handle_se2
6     JMP      #.loop
```

Medicine Cabinet

The Medicine Cabinet

Event System Quick Reference

Configure Events:

```
1      SETSE1/2/3/4  %#MMM_PPPPPP    ' Mode and pin
```

Event Modes:

%MMM	Trigger
%001	IN rises (Smart Pin ready)
%010	IN falls
%011	IN changes
%110	Pin rises
%111	Pin falls

Wait (blocking):

```
1      WAITSE1/2/3/4    ' Sleep until event
2      WAITCT1/2/3      ' Sleep until timer
3      WAITATN           ' Sleep until attention
```

Poll (non-blocking):

```
1      POLLSE1/2/3/4 WC ' Check event, clear flag, C=occurred
2      POLLCT1/2/3 WC   ' Check timer, C=reached
3      POLLATN WC       ' Check attention, C=received
```

Timer Setup:

```
1      ADDCT1/2/3 target, #delta    ' Set comparison target
```

Inter-COG:

```
1      COGATN #mask    ' Signal COGs (bit per COG)
```

Your Turn

Your Turn

Exercise 1: Event-Driven Serial

Rewrite a serial receive loop to use events instead of polling:

1. Configure SE1 for UART RX Smart Pin ready
2. Use WAITSE1 instead of TESTP loop
3. Measure the cycle count difference

```
1 ' Your code here:  
2 ' Hint: setse1 #%001<<6 + RX_PIN
```

Your Turn

Exercise 2: Dual Event Monitor

Create a loop that monitors both a button (pin edge event) and a timer (periodic event):

1. SE1 = button press (rising edge)
2. CT1 = 1 second heartbeat
3. On button: toggle LED
4. On timer: print timestamp

```
1 ' Your code here:  
2 ' Use POLL for both, handle whichever fires
```

Continue to Chapter 16: Multi-COG Orchestration →

Chapter 16: Multi-COG Orchestration

Bringing it all together in parallel harmony

The Hook: A Complete System in 8 COGs

Watch this system architecture come alive:

```

1  ' Main orchestrator (COG 0)
2  main_orchestrator
3      ' Launch the orchestra (SETQ sets PTRA for new COG)
4      SETQ    @sensor_params
5      COGINIT #1, @sensor_cog
6      SETQ    @motor_params
7      COGINIT #2, @motor_cog
8      SETQ    @comms_params
9      COGINIT #3, @comms_cog
10     SETQ    @display_params
11     COGINIT #4, @display_cog
12     SETQ    @safety_params
13     COGINIT #5, @safety_cog
14     SETQ    @logger_params
15     COGINIT #6, @logger_cog
16     SETQ    @debug_params
17     COGINIT #7, @debug_cog
18
19     ' Now coordinate them all
20     orchestrate
21         RDLONG  sensor_data, ##SENSOR_MAILBOX wz
22     if_nz CALL  #process_sensor_data
23
24         RDLONG  command, ##COMMAND_MAILBOX wz
25     if_nz CALL  #execute_command
26
27         CALL    #update_system_state
28         WRLONG  state, ##STATE_MAILBOX
29
30         JMP     #orchestrate

```

Eight independent processors, each with a specific job, all working in perfect coordination. This is the true power of P2!

Communication Patterns

The Mailbox Pattern

The simplest and most common:

```

1  ' Producer COG
2  producer
3      ' Generate data
4      CALL    #calculate_result
5      WRLONG  result, ##MAILBOX_ADDR
6
7  ' Consumer COG
8  consumer
9      RDLONG  data, ##MAILBOX_ADDR wz
10     if_z JMP    #consumer          ' Wait for data
11     WRLONG  #0, ##MAILBOX_ADDR     ' Clear mailbox
12     CALL    #process_data

```

The Ring Buffer Pattern

For streaming data between COGs:

```

1  ' Writer COG
2  writer_cog
3      RDLONG  wr_ptr, ##WRITE_PTR
4      WRLONG  data, wr_ptr
5      ADD     wr_ptr, #4
6      AND     wr_ptr, ##BUFFER_MASK ' Wrap around
7      WRLONG  wr_ptr, ##WRITE_PTR
8
9  ' Reader COG
10 reader_cog
11     RDLONG  rd_ptr, ##READ_PTR
12     RDLONG  wr_ptr, ##WRITE_PTR
13     CMP     rd_ptr, wr_ptr wz

```

```

14     if_z JMP      #reader_cog          ' Buffer empty
15
16         RDLONG    data, rd_ptr
17         ADD       rd_ptr, #4
18         AND       rd_ptr, ##BUFFER_MASK
19         WRLONG    rd_ptr, ##READ_PTR

```

The Command Queue Pattern

For sending commands between COGs:

```

1  ' Command structure in hub
2  ' +0: Command ID
3  ' +4: Parameter 1
4  ' +8: Parameter 2
5  ' +12: Result/Status
6  ' Commander COG
7  send_command
8      WRLONG    cmd_id, ##CMD_BUFFER+0
9      WRLONG    param1, ##CMD_BUFFER+4
10     WRLONG    param2, ##CMD_BUFFER+8
11     WRLONG    ##$FFFF, ##CMD_BUFFER+12 ' Mark as pending
12
13 wait_complete
14     RDLONG    status, ##CMD_BUFFER+12
15     CMP      status, ##$FFFF wz
16     if_z JMP      #wait_complete
17
18 ' Worker COG
19 process_commands
20     RDLONG    status, ##CMD_BUFFER+12
21     CMP      status, ##$FFFF wz
22     if_nz JMP      #process_commands ' No command
23
24     RDLONG    cmd_id, ##CMD_BUFFER+0
25     RDLONG    param1, ##CMD_BUFFER+4
26     RDLONG    param2, ##CMD_BUFFER+8
27

```



```

28          CALL    #execute_command
29          WRLONG  result, ##CMD_BUFFER+12    ' Signal complete

```

Synchronization Techniques

Using Locks

When multiple COGs need atomic access:

```

1  ' Atomic increment using lock
2  atomic_increment
3      LOCKTRY #COUNTER_LOCK wc
4      if_c JMP      #atomic_increment    ' Retry if busy
5
6      RDLONG  value, ##COUNTER
7      ADD     value, #1
8      WRLONG  value, ##COUNTER
9
10     LOCKREL #COUNTER_LOCK

```

Event Synchronization

COGs waiting for specific events:

```

1  ' COG 1: Signal event
2      WRLONG  ##EVENT_FLAG, ##EVENT_ADDR
3
4  ' COG 2: Wait for event
5  wait_event
6      RDLONG  flag, ##EVENT_ADDR wz
7      if_z JMP      #wait_event
8      WRLONG  #0, ##EVENT_ADDR    ' Clear event

```

Real-World Example: Robot Controller

Let's build a complete robot control system:

```

1  ' COG 0: Main Controller
2  main_controller
3      CALL    #init_system
4
5  main_loop
6      ' Read sensor hub
7      RDLONG  distance, ##DISTANCE_SENSOR wz
8      if_z JMP    #too_close
9
10     ' Check for commands
11     RDLONG  cmd, ##SERIAL_COMMAND wz
12     if_nz CALL  #process_command
13
14     ' Update motor speeds
15     CALL    #calculate_motion
16     WRLONG  left_speed, ##LEFT_MOTOR
17     WRLONG  right_speed, ##RIGHT_MOTOR
18
19     JMP     #main_loop
20 ' COG 1: Ultrasonic Sensor
21 sensor_cog
22     ' Trigger ultrasonic pulse
23     DRVH    #TRIGGER_PIN
24     WAITX   ##1000
25     DRVL    #TRIGGER_PIN
26
27     ' Measure echo time - wait for rising edge
28 .wait_hi
29     TESTP   #ECHO_PIN wz
30     if_nz JMP    #.wait_hi
31     GETCT   start_time
32 .wait_lo                                     ' Wait for falling edge
33     TESTP   #ECHO_PIN wz
34     if_z JMP    #.wait_lo
35     GETCT   end_time
36
37     ' Calculate distance
38     SUB     end_time, start_time
39     ' Convert to distance...

```

```

40      WRLONG  distance, ##DISTANCE_SENSOR
41
42      WAITX   ##10_000_000          ' 50ms at 200MHz
43      JMP     #sensor_cog
44  ' COG 2: Left Motor Driver
45  left_motor_cog
46      RDLONG  speed, ##LEFT_MOTOR wz
47      if_z JMP     #left_motor_cog    ' No speed set
48
49      ' Generate motor control signals
50      ' ... PWM generation code
51      JMP     #left_motor_cog
52  ' COG 3: Right Motor Driver
53  ' (Similar to left motor)
54  ' COG 4: Serial Communications
55  serial_cog
56      ' Check for incoming commands
57      TESTP   #RX_PIN wc
58      if_nc JMP     #serial_cog
59
60      CALL    #receive_byte
61      ' Build command...
62      WRLONG  command, ##SERIAL_COMMAND
63      JMP     #serial_cog
64  ' COG 5: LED Status Display
65  status_cog
66      RDLONG  system_state, ##STATE_MAILBOX
67
68      ' Display state on LEDs
69      CMP     system_state, #STATE_RUNNING wz
70      if_z DRVH  #GREEN_LED
71      if_nz DRVL #GREEN_LED
72
73      CMP     system_state, #STATE_ERROR wz
74      if_z DRVH  #RED_LED
75      if_nz DRVL #RED_LED
76
77      WAITX   ##10_000_000
78      JMP     #status_cog

```

```
79 ' COG 6: Safety Monitor
80 safety_cog
81     ' Monitor critical systems
82     RDLONG  battery, ##BATTERY_VOLTAGE
83     CMP     battery, ##MIN_VOLTAGE wcz
84     if_b WRLONG  ##STATE_SHUTDOWN, ##STATE_MAILBOX
85
86     ' Check temperature
87     RDLONG  temp, ##TEMPERATURE
88     CMP     temp, ##MAX_TEMP wcz
89     if_a WRLONG  ##STATE_OVERHEAT, ##STATE_MAILBOX
90
91     JMP     #safety_cog
92 ' COG 7: Debug Output
93 debug_cog
94     ' Output system state for debugging
95     ' ... debug code
```

Eight COGs, each doing one job perfectly, creating a responsive, reliable robot!

Your Turn: Multi-COG Project

Your Turn

Exercise: Traffic Light Controller

Requirements:

- COG 0: Main sequencer
- COG 1: North-South lights
- COG 2: East-West lights
- COG 3: Pedestrian button watcher
- COG 4: Timer/scheduler

Starting structure:

```

1      ORG      0
2  ' COG 0: Main sequencer
3      ' Launch other COGs
4      ' Coordinate light changes
5      ' Handle pedestrian requests
6
7  ' Your implementation here

```

Goal: Working traffic light with pedestrian crossing Hint: Use mailboxes for COG communication Success Check: Lights change correctly, pedestrian button works

The Medicine Cabinet

Multi-COG systems overwhelming? Start simple:

Start with just 2 COGs:

```

1  ' Main + Helper pattern
2  main      SETQ      @params          ' PTR for new COG
3          COGINIT #1, @helper
4          ' Main work
5  helper    ' Support work

```

Use simple mailboxes:

```

1  ' Fixed hub addresses for communication
2  MAILBOX_1 = $1000
3  MAILBOX_2 = $1004

```

Debug one COG at a time: Test each COG in isolation before combining!

Design Principles for Multi-COG Systems

1. **Single Responsibility:** Each COG does ONE thing well
2. **Loose Coupling:** COGs communicate through hub, not direct dependencies
3. **Clear Ownership:** Each piece of data has one writer
4. **Predictable Timing:** Real-time tasks get dedicated COGs
5. **Graceful Degradation:** System continues if one COG fails

Common Multi-COG Gotchas

1. **Race conditions** - Use locks for shared write access
2. **Deadlocks** - Avoid circular dependencies
3. **Starvation** - Ensure all COGs get resources
4. **Communication overhead** - Don't over-communicate
5. **Debugging complexity** - Use LED indicators for each COG

What We've Learned

You've mastered multi-COG orchestration:

- Communication patterns (mailbox, ring buffer, queue)
- Synchronization techniques
- Real-world system architecture
- Design principles
- Common pitfalls and solutions

This is it - you now understand the full power of the Propeller 2!

Your Journey Continues

You've completed this manual, but your P2 journey has just begun:

1. **Build something amazing** - Put your knowledge to work
2. **Share with the community** - Your projects inspire others
3. **Explore other manuals** - Smart Pins, Video, I/O await
4. **Push boundaries** - P2 can do things we haven't imagined yet

Chapter Summary

Congratulations! You've mastered multi-COG orchestration!

You now understand:

- How to coordinate 8 parallel processors
- Communication patterns between COGs
- Synchronization techniques
- Real-world system design

You did it! You're now fluent in PASM2 and ready to build incredible parallel systems!

Have Fun!

Remember what you've learned:

- Eight COGs working together are more powerful than any interrupt-driven system
- Parallel processing isn't harder, it's different
- The P2 way is about determinism and elegance
- Every complex system is just simple parts working together

Now go forth and create something amazing with your Propeller 2!

Epilogue: The Journey Forward

Well, here we are at the end... or should I say, at the beginning?

You've traveled from blinking your first LED to orchestrating eight parallel processors. You've mastered CORDIC mathematics, tamed the FIFO, and learned why interrupts are usually the wrong answer. That's quite a journey!

But here's the secret: everything you've learned is just the foundation. The P2 community continues to discover new techniques, new optimizations, new ways to use this remarkable chip. Every project pushes the boundaries a little further.

What Makes You Different Now

You're not just another embedded programmer anymore. You think in parallel. You see solutions that others miss. When someone says "that's impossible in real-time," you know better - you just dedicate a COG to it.

The Community Awaits

The Parallax forums are filled with fellow travelers on this journey. Share your projects. Ask questions. Help newcomers. The community that inspired this manual continues to grow because people like you contribute back.

One Last Story

I remember my first P2 project. I was trying to control 16 servos with perfect timing while reading sensors and communicating over serial. On my previous microcontroller, it was a nightmare of interrupts and jitter.

On the P2? Three COGs. Clean, simple, perfect timing. That's when I truly understood - this isn't just a different processor, it's a different philosophy of computing.

Your Challenge

Build something that wouldn't be possible without parallel processing. Something that would be a nightmare of interrupts on other processors. Then share it with the world.

Show them what eight COGs can do.

Show them the Propeller way.

"The best way to predict the future is to invent it."

— Alan Kay

And with your Propeller 2, you have everything you need to invent amazing futures.

Have Fun!

— The ghosts of deSilva, the P2 community, and one very enthusiastic AI

P.S. - Don't forget to blink an LED once in a while, just for old times' sake. It's still magical, even after all you've learned.

THE END

(But really, just the beginning...)

Further Reading

This teaching manual focuses on concepts, patterns, and building your understanding. For complete technical specifications, refer to these companion documents:

Propeller 2 Assembly Language (PASM2) Manual Complete PASM2 instruction details including syntax, timing, and flag effects for all 300+ instructions. Quick lookup reference for day-to-day development.

Parallax Propeller 2 Documentation (*v35, Rev B/C silicon, 2021-05-18*) Official silicon documentation from Parallax covering hardware specifications, electrical characteristics, and detailed register maps.

Appendix A: Platform Comparison

How P2 compares to other microcontrollers

If you're coming from another embedded platform, this appendix shows how the P2's approach differs and when those differences matter.

The Landscape

The embedded world is dominated by a handful of architectures:

Platform	Architecture	Typical Cores	Peripherals	Timing
STM32	ARM Cortex-M	1-2	Fixed location	Cache-dependent
ESP32	Xtensa/RISC-V	2	Fixed location	FreeRTOS scheduled
Arduino/AVR	AVR	1	Fixed location	Deterministic but slow
PIC32	MIPS	1	Fixed location	Interrupt-driven
P2 Propeller	Custom	8	Any pin	Deterministic

What Makes P2 Different

Eight Real Processors, Not Time-Slicing

On ARM, ESP32, or PIC, you typically have 1-2 cores that share time between tasks using interrupts or an RTOS. The P2 gives you eight complete, identical processors that run truly in parallel.

Traditional approach:

```

1 ' Everyone fights for the same CPU
2 ISR(TIMER1_vect) { motor_control(); } ' Might delay...
3 ISR(UART_RX_vect) { serial_handler(); } ' ...this
4 main() { while(1) { sensor_loop(); } } ' Hope we get time

```

P2 approach:

```

1 ' Each task owns its own processor
2 COG0: COGINIT(1, @motor_control) ' Coordinator launches workers
3 COG1: motor_control() ' Dedicated - always on time
4 COG2: serial_handler() ' Dedicated - never misses byte

```

```

5 COG3: sensor_loop()           ' Dedicated - consistent sample
6 COG4-7: ready for more

```

No interrupt priority juggling. No RTOS configuration. Each task owns its processor.

Smart Pins: Peripherals on Every Pin

Traditional MCUs have fixed peripheral assignments: UART1 is on PA9/PA10, SPI1 is on PB3/PB4/PB5, and if you need those pins for something else, you're stuck rerouting your PCB.

On P2, every pin contains a programmable state machine. Any pin can become a UART, SPI, PWM, ADC, quadrature decoder, or 27 other modes. The peripheral comes to your pin, not the other way around.

Deterministic Timing

ARM MCUs with cache have unpredictable timing. A memory read might take 1 cycle (cache hit) or 50+ cycles (cache miss). Even instruction timing varies—ARM instructions take 1-3+ cycles depending on the operation. This makes cycle-accurate timing extremely difficult.

P2 takes a different approach: nearly all instructions execute in exactly **2 clock cycles**. Want to know how long a code sequence takes? Count the instructions and multiply by 2. Hub memory uses round-robin access that gives every COG predictable, guaranteed access slots. Your timing loops work identically every time—no cache luck required.

Coming From ARM/STM32

You're used to configuring HAL structures, writing interrupt handlers, and managing DMA. Here's how P2 solves those problems:

Instead of...	On P2...	The Benefit
<code>HAL_UART_Transmit()</code>	Configure Smart Pin once, then WYPIN bytes	Pin handles all timing autonomously
<code>HAL_TIM_PWM_Start()</code>	Configure Smart Pin once, update with WYPIN	Pin runs independently—your COG is free
NVIC priority configuration	Nothing needed	All COGs equal, no priority inversion ever

Instead of...	On P2...	The Benefit
<code>HAL_DMA_Start()</code>	Use built-in FIFO/Streamer	Simpler API, integrated into each COG
<code>arm_sin_f32()</code> library	QROTATE instruction	Hardware trig in exactly 55 clocks
FreeRTOS <code>xTaskCreate()</code>	COGINIT	True parallel execution, not scheduled

The result: Deterministic timing, zero interrupt conflicts, and I/O configuration that just works.

Coming From ESP32

You're used to WiFi/Bluetooth convenience and FreeRTOS abstractions. P2 takes a different approach:

ESP32 Way	P2 Way	The Benefit
Built-in WiFi/BT	Add WizNet or ESP module	You choose your connectivity—or skip it entirely
<code>xTaskCreate()</code>	COGINIT	Not scheduled—truly parallel, guaranteed timing
GPIO matrix routing	Smart Pins	32 modes per pin, far more capability
FreeRTOS timing	Deterministic hub	Cycle-accurate timing guaranteed
Arduino framework	Spin2/PASM2	Deeper control, deeper understanding

The result: 8 real cores running simultaneously, timing you can count on, I/O flexibility that eliminates peripheral conflicts.

Coming From Arduino/AVR

You'll find P2 familiar but dramatically more powerful:

Arduino Way	P2 Way	The Upgrade
<code>digitalWrite()</code>	DRVH/DRVL or Smart Pins	Similar syntax, vastly more capability
<code>delay()</code> blocks everything	WAITX or dedicated COG	Timing without blocking other tasks
One thing at a time	8 things truly parallel	Real concurrency, not fake multitasking
8-bit math limits	32-bit + hardware CORDIC	No more overflow worries, hardware trig
Libraries for everything	Growing ecosystem + OBEX	More control, deeper understanding

The result: Graduate from 8-bit limitations to 8 parallel 32-bit processors with hardware math and Smart Pins on every I/O.

When P2 Is the Right Choice

P2 excels when you need:

- **Multiple real-time tasks** running simultaneously without conflicts
- **Precise timing** that cache misses and interrupts can't disrupt
- **Video or audio generation** requiring cycle-accurate output
- **Flexible I/O** where any pin can become any peripheral
- **Hardware math** for motor control, signal processing, or robotics
- **Multiple motor/servo control** with dedicated COGs per channel
- **Protocol implementation** where Smart Pins handle timing autonomously

Platform Trade-offs

Every platform makes trade-offs. P2 optimizes for **determinism, parallelism, and flexibility** rather than:

If you need...	P2's answer
Built-in WiFi/Bluetooth	Add WizNet or ESP module—you choose connectivity
Massive library ecosystem	Growing OBEX + helpful community

If you need...	P2's answer
Ultra-low-power sleep	External modules or different platform
Lowest unit cost at 100K+ volumes	P2 targets flexibility over commodity pricing

The honest reality: If your project is “connect to WiFi and display data,” an ESP32 does that with less effort. But if your ESP32 project is fighting timing jitter, missing deadlines, or running out of peripheral pins—that’s exactly what P2 solves.

Community Resources

While P2’s ecosystem is smaller than ARM or Arduino, it’s active and welcoming:

Parallax Forums - The heart of the P2 community. Chip Gracey (P2’s designer) participates actively, answering questions and discussing design decisions. You’ll find help from experienced developers who’ve solved problems you haven’t encountered yet.

P2 Object Exchange (OBEX) - A library of reusable Spin2 and PASM2 objects covering drivers, protocols, display interfaces, and more. Before writing something from scratch, check OBEX—someone may have already done the work.

Community Support - Unlike large platforms where your question disappears in a sea of posts, the P2 community is small enough that questions get noticed and answered. Many community members have decades of Propeller experience.

Coming from Arduino’s library-for-everything culture, you’ll write more code yourself—but you’ll understand it deeply, and help is always available when you get stuck.

The P2 Hardware Ecosystem

P2 isn’t just a chip - it’s a platform with expansion options:

Video & Audio:

- A/V Breakout Board: VGA, RCA, 80mW stereo, microphone input
- Digital Video Out: HDMI-type with differential signaling
- Built-in 8-bit DAC per pin (16-bit with dithering)

Connectivity:

- USB Host Board: Two USB-A ports

- USB Device Board: HID or CDC modes
- Serial Host/Device: RS-232 interfaces
- WizNet/ESP modules: Ethernet or WiFi

Development:

- P2 Eval Board: Complete development environment
- Edge Modules: 4MB or 32MB flash for embedding
- Breakout Boards: All 64 pins accessible

You add what you need - no paying for peripherals you won't use.

Summary

P2 represents a fundamentally different approach to embedded computing—one that eliminates entire categories of problems:

- **Eight processors** means your motor control never delays your serial handler
- **64 Smart Pins** means peripheral conflicts become impossible
- **Deterministic timing** means your code works the same way every time
- **Hardware CORDIC** means real-time math without floating-point libraries

Engineers who've fought interrupt priority inversions, missed timing deadlines, and PCB rework due to peripheral conflicts find P2 refreshing. You spend your time solving your actual problem, not fighting your MCU.

Welcome to the P2 community. You've got 8 processors, 64 Smart Pins, and a community that's been building amazing things since the original Propeller. Time to see what you can build.

Index

A

- ADC operations: Ch14
- ADD instruction: Ch3, Ch5
- ADDCT1/2/3: Ch15
- Address modes: Ch3
- ALTD/ALTS: Ch3
- Architecture: Ch2
- Assembly basics: Ch3

B

- Bit manipulation: Ch3
- Block transfers: Ch4, Ch9
- Booleans: Ch3

C

- C flag: Ch6
- CALL/RET: Ch3
- Clock timing: Ch2
- CMP instruction: Ch6
- COG anatomy: Ch2
- COG communication: Ch2, Ch16
- Conditional execution: Ch3, Ch6
- CORDIC: Ch7
- Counters: Ch2

D

- DAC operations: Ch14
- Debugging: Ch12
- Division: Ch5
- DRVH/DRVL: Ch1

E

- Egg beater: Ch2, Ch4
- Event system: Ch15

F

- FIFO: Ch4, Ch9
- Flags: Ch6
- Flow control: Ch3

G

- GETQX/GETQY: Ch5, Ch7

H

- Hardware multiply: Ch5
- Hub execution: Ch10
- Hub memory: Ch2, Ch4

I

- IN flag: Ch14, Ch15
- Immediate values: Ch3
- Instruction format: Ch3
- Interrupts: Ch11

J

- JMP instruction: Ch3

L

- LED control: Ch1
- Locks: Ch2, Ch16
- Logic operations: Ch3
- LUT memory: Ch13
- LUT sharing: Ch13

M

- Mailboxes: Ch2, Ch16
- Mathematics: Ch5
- Memory access: Ch4
- MOV instruction: Ch3

- MUL/MULS: Ch5
- Multi-COG: Ch16

O

- Optimization: Ch12

P

- Parallel processing: Ch2
- Pipeline: Ch7, Ch12
- Pins, Smart: Ch8, Ch14
- Platform comparison: Appendix A
- POLLSE1-4: Ch15
- PTR A/PTRB: Ch3, Ch4
- PWM: Ch8, Ch14

Q

- Q flag: Ch7
- QDIV: Ch5
- QROTATE: Ch7

R

- RDBYTE/RDWORD/RDLONG: Ch4
- RDLUT/WRLUT: Ch13
- RDPIN: Ch14, Ch15
- REP instruction: Ch3
- Rotation: Ch3, Ch7

S

- SETSE1-4: Ch15
- Shift operations: Ch3
- SKIP instruction: Ch3, Ch6
- Smart Pins: Ch8, Ch14
- Streamer: Ch9, Ch13

T

- Timer: Ch2
- Timing: Ch2, Ch12
- Trigonometry: Ch7

U

- UART: Ch8, Ch14

V**W**

- WAITSE1-4: Ch15
- WAITCT1-3: Ch15
- WAITX: Ch1
- WRBYTE/WRWORD/WRLONG: Ch4
- WRPIN/WXPIN/WYPIN: Ch14

Z

- Z flag: Ch6